PROTODUINO · DESIGN DOCUMENT

A streaming ELF loader for AVR

Functional and technical design of a compile-time configurable ELF32 loader: a bootloader that is one protothread, lent to the running system to load modules that are process types. With measurements and fixes for protoduino caveats.

Draft 1.1 · 14 November 2026

jklarenbeek/protoduino · branch ccr-8617b1c3-gjnqlx

1 thread

the bootloader is a Protothreads v2 thread, and the kernel schedules the same one

2,020 B

in the 2 KB boot section of an ATmega328P, with the thread offered to the kernel

15,113

turns a heartbeat process took while a module was loaded beside it

64 of 64

simulated scenarios behaved as designed, twenty-three of them in protosim itself

What is real in this document

- The loader was written and run for this document. Every figure marked MEASURED comes from it: avr-gcc 7.3.0, and simavr at the commit that protosim pins.
- Module loading runs on protoduino's scheduler, serial device process and pipes, on the ATmega328P and the ATmega2560. Since this draft it is also driven from a shell: a sketch built by the Arduino builder took modules over its command line inside protosim (9.2).
- Nothing has run on hardware. Power loss was not simulated. simavr programs a page in no time, so timings leave out the 7 to 9 ms a real page costs (8.1). Relocation (4.8) is a design; the ATtiny build compiles and has not executed.
- The loader is part of the library since this draft: the stream, the load thread and the flash driver in `src/`, module slots and the loader process as headers a sketch configures, the bootloader in `boot/`, the layout of a module in `ld/` (6.1). The test bench stays in `docs/elfloader/prototype/`. The library itself changed in a few places, listed in 7; most fixes of sections 7 and 8 are still proposals. No simavr source file was changed.

MEASURED built and run for this document READ established by reading source or tool output PROPOSED designed, not built BUILT compiles, not executed

Contents

1 Summary
2 Goals, non-goals and target parts
3 What earlier loaders got wrong
4 The ideas the design rests on
5 Functional specification
6 Technical design

7 Fixes for current protoduino caveats
8 Simulator findings: simavr and protosim
9 Verification
10 Roadmap, risks and open questions
A ELF fields the loader reads
B Reproducing the measurements · Sources

1 Summary

The loader takes the file the linker writes, the `.elf`, and treats it as a stream. The first 52 bytes say what the file is. The next 32 bytes per program header say which bytes belong where. Everything after that is delivered to flash in the order it arrives. The stream never seeks backwards and never allocates, and all of its state fits in one 84-byte structure. When the last loadable byte has been delivered it is done; the symbol tables and debug information that make up most of the file never have to be sent.

Around the stream sits one Protothreads v2 thread, `elf_load_thread()`. It receives the file with XMODEM-CRC, feeds the stream and programs the pages. It does no input or output of its own and reads no clock: bytes pass through a small mailbox in its workspace, and it returns to its caller between any two of them. That one thread is used in three places, chosen at compile time:

- **As the bootloader.** At reset nothing else exists, so a loop of a dozen lines schedules the thread from a polled `uart0`. Any terminal program can send the unmodified `.elf`. An image record written last decides whether the application may start.
- **As a boot ROM for the running system.** The boot section exports the address of the thread. A protoduino process schedules it with the bytes its pipe delivers, and every other process keeps its turn while a module arrives. The kernel pays for a process, not for a loader.
- **Inside the kernel.** Where the bootloader is Optiboot, or a protoduino bootloader built without the export, the kernel links its own copy of the thread and has pages programmed through whatever the bootloader offers.

Loading is therefore cooperative everywhere, and it is not a feature that a larger boot section switches on. The fuses decide only where the thread lives: in the boot section when it fits, in the kernel when it does not (4.6). The build fills the section that the fuses reserve with as many options as fit, in a set order of priority (6.9).

Modules fit protoduino in particular. A module is a process type: the same record `PROCESS_DEFINE` emits for a built-in process, linked into a flash slot and started with `process_new()`. Protothread processes keep their state in a struct the kernel hands them, so a module needs no static RAM, and the hardest part of dynamic loading on an AVR, relocating data addresses, does not arise.

Result	Value	Status
Stream state, ATmega328P build	84 bytes, of which 52 are header scratch	MEASURED
Bootloader, ATmega328P	1,930 B flash, 377 B RAM. With the thread offered to the kernel 2,020 B in the 2 KB boot section, 28 bytes spare. With every option 2,334 B, which needs the 4 KB section.	MEASURED
Bootloader, ATmega2560 and ATmega1284P	2,950 B with every option, which is how it is built: these parts take the 4 KB boot section, and everything fits there with 1,146 bytes to spare. (Without the options: 2,338 B.)	MEASURED
ATtiny85 build with serial on two pins	1,958 B flash, 313 B RAM	BUILT
Flash written by the bootloader against simavr's own ELF reader	Identical on the ATmega328P, ATmega1284P and ATmega2560, including a 106 KB image that reaches far flash	MEASURED
A file for the wrong core family, offered to a device with a working application	Refused with a reason code; nothing erased; the application starts again	MEASURED
Start-up: the bootloader waits for a sender only after a reset through the pin	After power-on, a brown-out or a watchdog reset the image starts at once; an upload tool or the reset button opens the window. Run in the harness and inside protosim, which records reset causes since `f4d382e`: a file uploaded over its PTY after a pin reset, no wait after a power cycle.	MEASURED

Result	Value	Status
An image that the watchdog resets, because it hung or to restart itself	Starts again at once after every reset and reads the reset cause the bootloader hands it. The previous draft left the watchdog alone and the image never came back.	MEASURED
A module sent to a running protoduino kernel, loaded by the thread in the 2 KB boot section	Programmed into its slot, started with `process_new()`, entered into the module table, and started again after a reset. A heartbeat process took 15,113 turns during the 350 ms transfer.	MEASURED
The same on the ATmega2560, where the thread sits in the 4 KB boot section above 128 KB	Works: the kernel reaches the thread with a far call that leaves `EIND` alone. 14,446 heartbeat turns during the 365 ms transfer.	MEASURED
The same with a stock Optiboot 8.3 as the bootloader	Works: the kernel links the thread and programs pages through Optiboot's `do_spm`	MEASURED
A module linked against another kernel build	Refused before a single page is programmed	MEASURED
A second module loaded beside a running one	Both run, and both start after a reset	MEASURED
A module that declares static RAM	Fails at link time with a clear message	MEASURED
The whole chain as a user meets it, inside protosim: bootloader, a sketch with a shell built by the Arduino builder, modules sent to the shell	The sketch arrives through the bootloader; `get` takes modules into flash slots while the other processes run, `del` removes them, a module for another build is refused and the log says why, and what a slot holds runs again after a power cycle. On the ATmega328P and the ATmega2560.	MEASURED

The size is the weak point, and it should be stated plainly. Optiboot fits in under 512 bytes; this bootloader is 1,930 bytes, in C with its page writer in assembly, built with an old compiler. It is 292 bytes larger than the blocking version of the previous draft (1,640 bytes). The protothread form cost 374; writing the page programmer the way Optiboot does gave 62 back and numbering the continuation points 14. Since then 56 bytes went into things the blocking version did not do: taking care of the watchdog, starting an image at once after a watchdog reset, and handing the image a UART in its reset state. They were paid for with the 62 bytes of segment merging, which a fourth entry in the segment table replaces. On parts with more than 64 KB of flash it does not fit the 2 KB boot section. That question is settled: those parts take the 4 KB section, 1.6 % of the flash of an ATmega2560, and the bootloader is built with every option there. What the bytes buy is in 4.6; where they go is in 6.10.

Building on the library also turned up thirteen more findings in protoduino itself, among them that `process_run()` lets polls starve events and that a process which ends is never cleaned up unless another event happens to reach it. Section 7 lists them with measurements and patches.

2 Goals, non-goals and target parts

Goals

- **Accept the linker's own output.** No hex file, no binary image and no packaging step for firmware.
- **One core, used in parts.** From a bootloader to a loader inside the kernel, with features that cost nothing when switched off.
- **Fill the boot section.** Whatever block the fuses reserve is used for as many features as fit.

Non-goals

- **Linking on the device.** Relocatable `.o` files, symbol tables and name lookups stay on the host.
- **Authenticated updates in the first version.** Integrity is checked; origin is not. The reserved byte of the module descriptor is where a signature would be announced (6.8).

- **Refuse before erasing.** The wrong file must be rejected while the installed image is still intact.
- **Always recoverable.** On parts with a hardware boot section, no failed update may leave a device that cannot be reached over the same serial line.
- **Modules are processes, loadable at run time.** Wherever the part can carry it and unless it is configured out. Three levels: none, modules already in flash, run-time loading.
- **Loading never stops the system.** The loader is a protothread among protothreads, and it reads like the rest of protoduino.
- **Not tied to our bootloader.** A protoduino kernel can load modules on top of Optiboot as well.
- **C++ runtime features in modules.** No exceptions, RTTI or static constructors.
- **Parts without room for a page buffer.** The ATtiny4/5/9/10 have 32 bytes of RAM.
- **UPDI-era parts in this document.** tinyAVR 0/1/2 and AVR Dx program flash through NVMCTRL. They need another flash back-end, which the layering allows but which is not designed here.

Part	Flash	RAM	Page	Boot section	Bootloader	Run-time modules
ATmega328P	32 KB	2 KB	128 B	0.5 to 4 KB	2 KB section MEASURED	Yes. The load thread fits the 2 KB section and the kernel schedules it there. MEASURED
ATmega2560	256 KB	8 KB	256 B	1 to 8 KB	4 KB section, every option MEASURED	Yes. The thread sits in the 4 KB section, above 128 KB, and the kernel reaches it with a far call (6.7). MEASURED
ATmega1284P	128 KB	16 KB	256 B	1 to 8 KB	4 KB section, every option MEASURED	As the ATmega2560, without the far call: its boot section is in reach of a plain call. The kernel side was not run. PROPOSED
ATtiny85	8 KB	512 B	64 B	none	BUILT	Off by default: the workspace of the thread alone is 313 of its 512 bytes of RAM. These parts get a different arrangement: the shell on the PC and a port on the part, which on the ATtiny85 also programs modules into flash slots, without a bootloader and without RAM for a page (9.2). MEASURED
ATtiny25 / 45	2 / 4 KB	128 / 256 B	32 / 64 B	none	Needs a small-frame transport PROPOSED	No. Excluded at compile time.
ATtiny4/5/9/10	≤ 1 KB	32 B	–	none	Out of scope	–

Flash end, RAM range, page size and core family were read from the avr-libc device headers. Boot-section sizes are from the datasheets and were not re-checked here. The request named an “ATtiny8”; no such part exists, so this document takes the ATtiny25/45/85 family as the small target, with the ATtiny85 as reference.

3 What earlier loaders got wrong

Five existing designs were examined, four of them in source. Each got something right that this design keeps, and each left something that it changes.

Loader	What it does, and where it falls short	What this design does instead
Contiki ELF loader 2005 to 2017 READ	Loads relocatable `.o` files and links them on the device. It makes 14 seek-and-read calls, recognises ten sections by comparing name strings, and resolves every import with `strcmp` against a table of names compiled into the kernel. So the whole file must already sit in a seekable store, and the kernel carries its symbol names in flash. It checks the first seven identification bytes and never the machine type. Contiki later added a pre-linked format (CLE), and Contiki-NG deleted the loader in 2017, 4,203 lines in one commit.	No linking on the device. The file is consumed as it arrives. Machine and core family are checked from the header. What Contiki got right is kept: a module is a set of processes, and architecture code is separate from the core.
Optiboot with avrdude	Under 512 bytes, which is the bar for size. But the device receives anonymous pages: it has no idea what the image is, keeps no record of whether it is complete, and starts whatever is there after a timeout. The host has to turn the ELF into a hex file and speak a programmer protocol. Version 8 added `do_spm()` so an application can write flash.	The device reads the ELF header itself and keeps an image record. The `do_spm()` idea is kept and widened: the boot section lends out its whole load thread (6.7). Optiboot also stays a supported choice: a protoduino kernel loads modules on top of it (6.8).
urboot, micronucleus parts without a boot section	They patch the application's reset vector so that reset reaches the loader, and rely on erased flash running through to it. The known hazard, in urboot's own words: if the first page is erased but not rewritten, the part is bricked until it meets an ISP programmer.	Erase from the top down with page 0 last, and give the loader a landing pad. There is then no instant at which a reset fails to reach the loader (6.6). PROPOSED The run-through and the landing pad were checked in simulation (T13).
simple-avr-bootloader m3y54m; a tutorial by its own description READ	Sends a raw binary with a 16-bit length. Frames carry an 8-bit additive checksum and no sequence number, and the uploader retries on timeout, so a lost acknowledgement makes the same chunk land twice. The update flag sits 8 bytes below the top of RAM, where the stack starts. The loader runs its serial port from interrupts and has no check that the application is whole.	XMODEM-CRC: 16-bit CRC and block numbers, so a repeat is acknowledged and dropped. A polled serial port with no vectors. An image record. Its good idea is kept: the application asks for an update, then resets.
Host ELF libraries libelf, ELFIO and others	They assume random access or the whole file in memory, and they allocate. None is built to be fed.	Used only as oracles in tests.
Zephyr LLEXT a current reference	Still links on the device, but can replace symbol-name strings with hashes to save space. That is the right direction, and still far too much code for an 8-bit part.	One step further: no symbol table on the device at all. A module is linked on the host against one firmware build and carries that build's identity (6.8).
protoduino `docs/loader.md` READ	Describes modules that are stored in RAM and run from there, which an AVR cannot do: code executes only from flash. It compiles with `-fpic`, which avr-gcc reports as unsupported. It refers to functions that are not in `src/`. Its file header has no magic number, version or checksum.	Superseded by this document.

An earlier note of mine questioned the order in which simple-avr-bootloader fills, erases and writes a page. That was wrong: the datasheet allows the page buffer to be filled either before or after the erase.

4 The ideas the design rests on

4.1 The ELF file is the wire format

An executable from avr-gcc is laid out in the order a loader needs it. The ELF header comes first, the program headers follow at offset 52, the loadable bytes come next, and everything a loader has no use for comes last. A reader that only moves forward can therefore do the whole job, and can stop early.

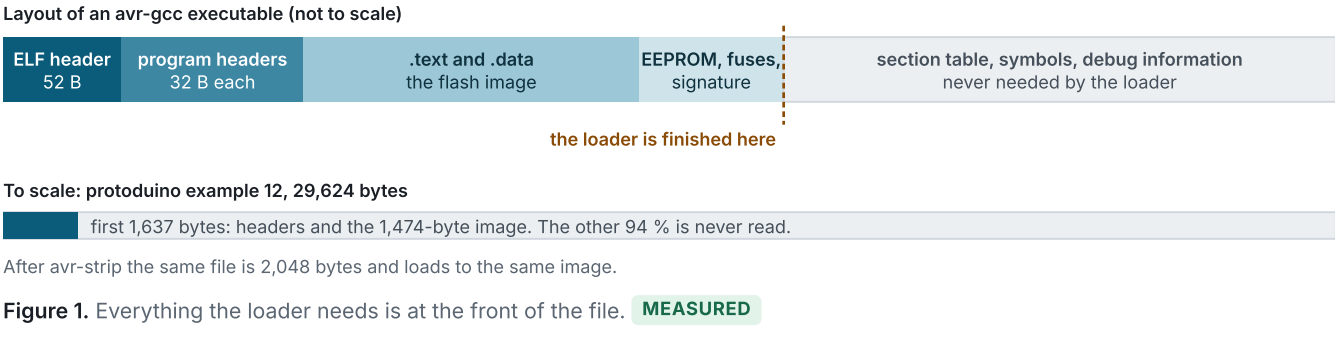


Figure 1. Everything the loader needs is at the front of the file. MEASURED

File	File size	Loader finished within	Result against `avr-objcopy`
Test program, ATmega328P, with EEPROM data, fuses and signature	12,868 B	660 B	Flash and EEPROM identical
protoduino example 12 (UART echo), ATmega328P	29,624 B	1,637 B	Identical, also when fed one byte at a time
106 KB test image, ATmega2560	117,616 B	106,986 B	Identical

The consequence for the user is that no tool has to be installed to flash a board. With XMODEM as the transport (6.4), a terminal program that every platform already has can send the file the build produced.

4.2 One core, used in parts

The stream is a state machine with no input or output of its own. Bytes are pushed in from whatever carries them; bytes that belong in a memory are handed to a sink. Around it sits one protothread, the code that receives a file and feeds the stream, written so that all of its state is in a workspace the caller owns. That makes the thread usable from three places: the bootloader at reset, the boot section scheduled by the running kernel, and a kernel that links it in.

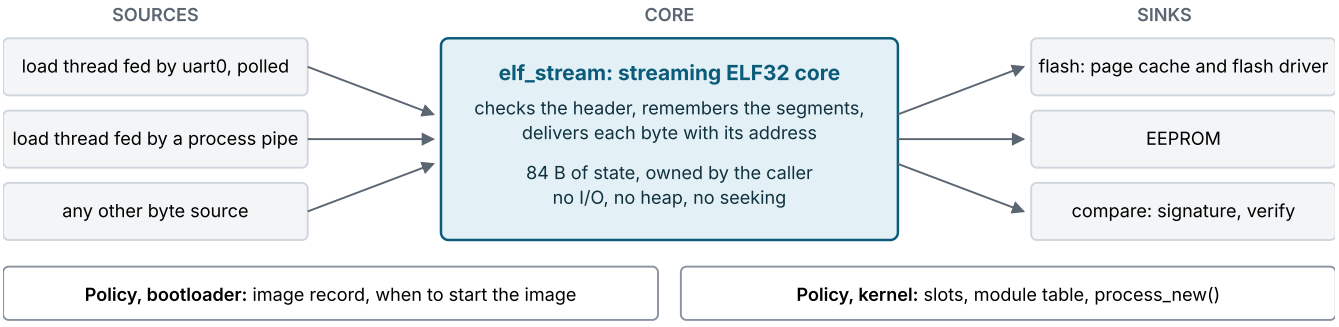


Figure 2. The pieces. All of them are built except the third source; the compare sink covers the signature, not yet a verify mode.

Which parts end up where depends on two choices: what sits in the boot section, and how much module support the kernel is built with.

In the boot section	Kernel built for	Who runs the load thread	Tested
protoduino bootloader that offers its load thread	run-time loading, thread of the bootloader	The loader process schedules the thread inside the boot section	M1, M9
protoduino bootloader with only the page service	run-time loading, own thread	The loader process schedules the kernel's copy; pages go through the page service	M3
Optiboot 8 or later	run-time loading, own thread	The same; pages go through `do_spm`	M2
anything	modules already in flash	Nobody. Modules are programmed with the firmware and only started.	M4
anything	no modules	–	M8

4.3 Address spaces select the sink

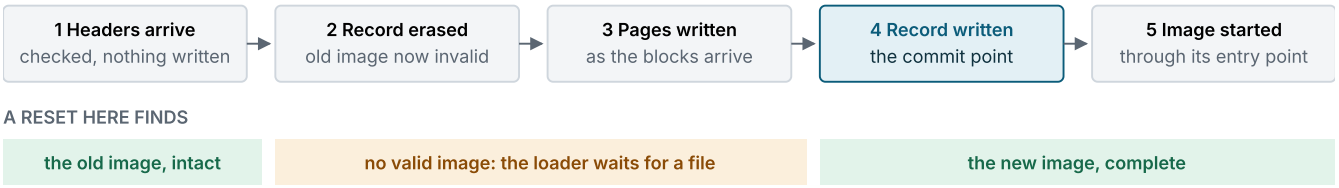
avr-gcc gives every memory of the chip its own range in the ELF address space, so the top byte of a segment's load address says where its bytes belong. The loader routes on that byte. It uses the load address (`p_paddr`), not the run address: the initial values of `.data` are stored in flash right after the code, and the program's own start-up copies them to RAM.

Load address	Memory	What the loader does	Option
`0x00....` upward	Flash	Writes, inside the permitted window only	always
`0x80....`	RAM	Nothing to write. `.bss` has no file bytes.	–
`0x81....`	EEPROM	Writes the EEPROM image	`ELF_CONF_EEPROM`
`0x82....`, `0x83....`	Fuses, lock bits	Ignores; software cannot write them	–
`0x84....`	Signature	Compares with the chip's own signature bytes	`ELF_CONF_SIGNATURE`

Two checks come free with the format. The header's flags name the core family the file was built for (`avr:5` for an ATmega328P, `avr:6` for an ATmega2560, `avr:25` for an ATtiny85), and avr-gcc defines the same number as `__AVR_ARCH__` when the loader is compiled. One comparison rejects a file for the wrong family from its first 52 bytes. The signature check is finer, down to the exact device, but it comes late, because avr-gcc places the signature after the flash data (see FR-9).

4.4 Refuse before erasing, commit last

An update is a transaction with one commit point. Nothing in flash changes until every program header has been accepted. The installed image is declared invalid before its first byte changes, and becomes valid again only when a small record is written after the last byte. A reset at any moment therefore finds either the old image, or no valid image and a waiting loader, or the new image.



A refusal in step 1 leaves the device exactly as it was. The first prototype got this wrong; test T7 found it.

Figure 3. The update as a transaction. Steps and refusals were exercised in simulation; a power cut was not. MEASURED

4.5 The boot section as a small ROM

On a megaAVR the SPM instruction works only from the boot section, so an application cannot program flash by itself, and a run-time loader in the application cannot exist without help. The boot section therefore exports a table at a fixed place at the very end of flash. Its first entry programs one page from the caller's buffer, as

Optiboot's `do_spm()` does. Its second is the address of the load thread itself: the same code that loaded the firmware at reset, to be scheduled by the running system on a workspace the system owns. The boot section is paid for anyway; this makes it do the kernel's loading too (6.7). MEASURED

4.6 One thread, two schedulers

The previous draft exported a blocking session: the kernel called into the boot section and got control back seconds later, when the module was in flash. This draft exports a protothread, and that changes what the kernel can do while a module arrives.

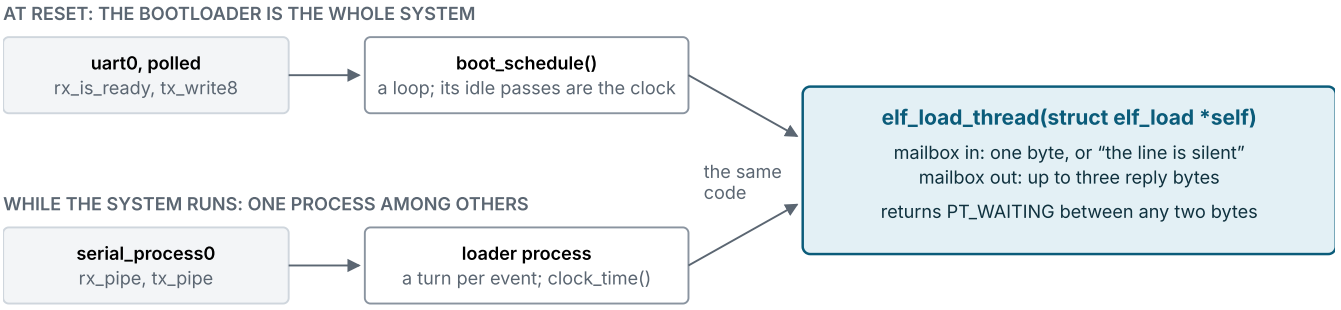


Figure 4. The load thread does no I/O and reads no clock, so the bootloader and a process can both drive it. In M1 the lower path ran the thread where it sits in the 2 KB boot section of an ATmega328P, in M10 in the 4 KB section of an ATmega2560.

What was measured. A sketch on protoduino's scheduler ran the serial device process, a kernel process, the loader process and a heartbeat process that takes a turn as often as it can. While a 3,144-byte module file arrived in 350 ms, the heartbeat took 15,113 turns. With the blocking session of the previous draft it would have taken none. MEASURED

How long the others wait. The longest wait of the heartbeat was 15.2 ms, the time one XMODEM block spends on the wire. That is not the loader: `process_run()` serves polls before events, and the serial device polls on every received byte, so an event-driven process waits out the block (7, finding 14). Built against a copy of `process.c` with that one function changed, the longest wait fell to 2.0 ms, which is the time the thread needs to check a block and push it through the stream. MEASURED On a chip, each page adds 7.4 to 9.0 ms: an AVR cannot execute application code while it erases and programs application flash, and interrupts stay off for that time. No design removes that; XMODEM's stop-and-wait keeps the sender quiet meanwhile.

Is cooperative loading tied to a fuse setting?

No. There is one implementation of the load, the thread, so every build that can load at all loads cooperatively. The size of the boot section decides only where the thread lives:

- **ATmega328P, 2 KB section.** The bootloader with the thread exported is 2,020 bytes. The kernel schedules the thread in the boot section and links 1.7 KB for the loader process and module table.
- **ATmega328P, 4 KB section.** The same, plus every other option, in 2,334 bytes.
- **ATmega1284P and ATmega2560, 4 KB section.** The standard configuration for these parts: every option, the thread included, in 2,950 bytes. The kernel schedules the thread there, on the ATmega2560 through a far call.
- **Optiboot, or a section that is too small for the export.** The kernel links its own copy of the thread, 3.1 KB in all, and loads just as cooperatively.

The build step of 6.9 makes the choice from the block size. `BOOT_CONF_LOAD_SERVICE` forces it either way, and `LOADER_CONF_THREAD` tells the kernel which thread to schedule.

4.7 A module is a process type

protoduino already has the right descriptor. `process_type_t` is a small record in flash holding a name, a thread function, an instance size and a priority, and `process_new(type, storage)` turns it into a running process in memory the caller supplies. A loadable module is exactly that record plus its code, linked into a flash slot. Its source is an ordinary process, written with `PROCESS_DEFINE` and `PROCESS_THREAD`, with one more line: `MODULE(name)`. Loading a module means programming its bytes and calling the function every built-in process type is started with. MEASURED

Protothreads are stackless, and a protoduino process keeps its variables in its instance struct. A module written that way has no `.data` and no `.bss`, so there is nothing in RAM to place or relocate: the kernel hands the thread a pointer, and the module works wherever that points. The rule is enforced at link time rather than left to discipline (6.8).

4.8 Relocation without a relocator

A module is linked for one slot, which needs no code on the device. To let one file load into any slot, addresses inside the module must be adjusted, and the usual way needs a relocation engine. There is a cheaper way. If slots are aligned to 512 bytes, moving a module changes only the high byte of every byte address and of every word address; low bytes never change and no carry crosses between them. A fix-up then is one position in the stream whose high byte gets a constant added. The host sorts those positions by offset and puts the list ahead of the code, and the loader applies them as the bytes go by, still in one pass. On parts with absolute call instructions, calls from the module into the kernel need no fix-up at all. PROPOSED

4.9 The same core on the host, and the simulator as oracle

The stream is plain C with no AVR dependency, so it compiles unchanged for the host. That gives three things: unit tests and fuzzing at desktop speed; a host tool that can say exactly what a device will accept, because it runs the device's code; and, compiled to WebAssembly, the same check inside a Node command-line tool or protosim's web front end. For end-to-end tests, the simulator already contains a trusted ELF reader. Flashing a file through the loader inside simavr and comparing the result with what simavr's own reader produces for the same file tests the loader against an independent implementation. Section 9 reports that comparison.

5 Functional specification

5.1 Use cases

ID	Use case	How it goes	Status
UC1	Flash firmware over a serial line	Reset the board. The bootloader asks for a file by sending `C`. Send the `.elf` with XMODEM from any terminal program. The bootloader programs it, stores the record and starts it.	MEASURED
UC2	Recover a board whose application is broken or half-written	The record is missing or damaged, so after reset the bootloader keeps asking for a file. No programmer is needed on parts with a boot section.	MEASURED
UC3	The application asks for an update	It sets a flag that survives reset and resets through the watchdog. The bootloader sees the flag and waits for a file.	PROPOSED
UC4	Check what is installed	Send the same file in verify mode. Bytes are compared with flash instead of programmed.	PROPOSED
UC5	Ask the device what it is	Send `?` before a transfer. The bootloader answers with a line that names it.	BUILT

ID	Use case	How it goes	Status
UC6	Load a module while the system runs	A process starts the loader process for a free slot. The loader schedules the load thread with the bytes from the serial device process, while every other process keeps running. It then checks the module, enters it into the module table, starts it with `process_new()` and tells whoever asked.	MEASURED
UC6b	Run with modules that are already in flash	A kernel built without the loader starts the modules its module table lists. They were programmed with the firmware or loaded earlier.	MEASURED
UC7	List, stop and remove modules	Commands of the shell (`src/mod/shell.h`): `mods` , `stop` , `kill` , `get` , `del` . Removal is `module_table_write(slot, false, ...)` and `process_destroy()` .	MEASURED U1, U3, U4

5.2 Requirements

ID	The loader ...	Evidence
FR-1	accepts ELF32, little-endian, executable files for the AVR machine type whose program headers start at offset 52, fed in file order. This is what avr-gcc emits.	MEASURED T1, T3, T11, host
FR-2	needs nothing after the last loadable byte. Whatever follows, including XMODEM padding, is ignored.	MEASURED T3, T4
FR-3	refuses a file built for another core family from the ELF header alone.	MEASURED T5, T7
FR-4	refuses any flash segment outside the permitted window. The bootloader's own pages and the image record lie outside the window.	MEASURED M6, host
FR-5	changes nothing in flash until every program header has been accepted.	MEASURED T7
FR-6	invalidates the installed image before changing its first byte, and makes an image valid only after its last byte is programmed.	MEASURED T1; power cut not simulated
FR-7	has a damaged block sent again, and acknowledges a repeated block without applying it twice.	MEASURED T2
FR-8	optionally writes the EEPROM image contained in the file.	MEASURED T8
FR-9	optionally compares the file's device signature with the chip's. A mismatch prevents the commit. The signature follows the flash data in the file, so by then flash and EEPROM have changed and the device stays in the bootloader. A host tool can make the same check before sending; `tools/talk.mjs` does not.	MEASURED T9
FR-10	after a refusal sends a one-byte reason from protoduino's error codes and is ready for a new transfer. If nothing was programmed, the old image still starts.	MEASURED T5, T6, T7
FR-11	starts an image only if its record is intact, and optionally only if the image's CRC still matches.	record MEASURED; CRC at start BUILT
FR-12	exports a service table to the running system: programming a page, and the load thread itself. Both use the caller's memory only.	MEASURED T10, M1, M3
FR-13	loads a module while the kernel runs, into the slot the kernel names and nowhere else; the kernel image is left untouched.	MEASURED M1 to M3, M6, M9
FR-14	lets the kernel start the modules of its module table again after a reset, without a loader.	MEASURED M1 to M4, M9
FR-15	refuses a module built against another kernel build before any page of it is programmed.	MEASURED M5: 0 bytes programmed
FR-16	works at three levels of module support chosen at compile time, and is off by default where the part is too small.	levels MEASURED M4, M8; defaults PROPOSED

ID	The loader ...	Evidence
FR-17	does not depend on the protoduino bootloader: on Optiboot 8 or later the kernel loads modules through `do_spm`.	MEASURED M2
FR-18	fills the boot section the fuses reserve with as many options as fit, in a set order.	MEASURED 6.9
FR-19	never has to accept a module with static RAM: the module linker script rejects it.	MEASURED
FR-20	never stops the system while a module arrives: the load is a protothread that returns between any two bytes, and the loader is a process like any other.	MEASURED M1 to M3, M9: heartbeat turns
FR-21	holds several modules, each in its own slot, and loads one beside others that run.	MEASURED M9
FR-23	survives the watchdog: an image that the watchdog resets starts again, and finds the reset cause where the bootloader left it.	MEASURED T22, T23
FR-24	lets the build choose after which resets it waits for a sender, and always waits for an image that jumps to it.	MEASURED T24 to T27
FR-22	uses the framework it lives in: Protothreads v2 for control flow, `ptstate_t` for results, `errors.h` for reasons, processes, pipes and events for the kernel side, and the `*.conf.h` pattern for settings.	READ 6.1 to 6.9

5.3 Start-up behaviour

After a reset the bootloader probes for a sender: it waits for about a second of silence, transmits `C`, and repeats. A transfer may begin at any probe. If none begins within the configured number of probes and the image record is intact, the image is started. If the record is not intact, the bootloader probes for ever. After a refusal the window opens again. In simulation the probes came 0.950 s apart, and with three probes the image started 3.8 s after reset. MEASURED

Which resets open the window is a setting, `BOOT_CONF_WINDOW_AFTER`: a mask of reset causes. By default only a reset through the pin does, which is what an upload tool and a reset button make, and the rule Optiboot uses. After power-on, a brown-out or a watchdog reset a complete image starts at once: a board is up without the 3.8 s, an image that the watchdog stopped is running again, and no probe goes out on a serial line that belongs to the application. A board without a reset line can add power-on and brown-out to the mask and is then updated by cycling its power; with all four causes the bootloader waits after every reset. An image that jumps to the bootloader is always waited for: that is how a running system asks to be replaced. The three settings and the jump were run in the harness (T23 to T27), and the default inside protosim (P1 to P5). MEASURED

The watchdog. An image that the watchdog resets, because it hung or because that is how it restarts itself, leaves the watchdog running at its shortest period, 16 ms, until software clears the cause and switches it off. A bootloader that ignores this is reset again long before its first probe, over and over, and the board stays dead until power is cycled. The previous draft of this bootloader did ignore it, and in simulation the image never came back (T22). It now deals with the watchdog before anything else and hands the reset cause to the image (6.5); the same image then restarts at once after every watchdog reset (T23). MEASURED

5.4 Reasons for refusal

When the load thread refuses a file it cancels the transfer and sends one more byte with the reason. The reason is the error the thread ends with, and it is a code from protoduino's `sys/errors.h`, so the error logger prints it by name.

Code	Name	Meaning	Flash changed?
`0x51`	`ERR_BIN_MAGIC`	Not a little-endian ELF32 executable for AVR with program headers at offset 52	no
`0xD0`	`ERR_BIN_VERSION`	Built for another AVR core family	no

Code	Name	Meaning	Flash changed?
`0x50`	`ERR_BIN_HEADER`	No program headers, or an entry address that does not fit	no
`0xA2`	`ERR_BOUNDS_LOWER`	A segment starts below the window: firmware offered to a module slot	no
`0xA3`	`ERR_BOUNDS_UPPER`	A segment ends above the window, or an offset or size does not fit the build's address width	no
`0x70`	`ERR_ORDER_SEQUENCE`	Segment data is not in ascending file order	no
`0xE0`	`ERR_BUF_FULL`	More separate segments than the build remembers MEASURED T29	no
`0xD3`	`ERR_VALID_REQUIRED`	Nothing to load	no
`0xA4`	`ERR_INIT_DEPENDENCY`	A module linked against another kernel build	no
`0xCA`	`ERR_PROTO_SEQUENCE`	A block was skipped BUILT	possibly; not committed
`0x53`	`ERR_VALID_CONSTRAINT`	The device signature in the file is not this chip's	yes; not committed
`0x61`	`ERR_BUF_UNDERFLOW`	The sender finished before the file was complete BUILT	possibly; not committed

The loader process adds three of its own, which reach the error logger and not the sender: `ERR_INIT_NOSUP` when the bootloader can neither program a page for the kernel nor offer a thread that fits, `ERR_STATE_LOCKED` when the slot holds a module, and `ERR_CONN_TIMEOUT` when nobody sent a file. After a load, `module_check()` can still say `ERR_BIN_MAGIC`, `ERR_BIN_VERSION` or `ERR_HEAP_OOM` (the process struct of the module does not fit the RAM a slot gets).

5.5 Limits

- At most 255 program headers; at most `ELF_CONF_MAX_SEGMENTS` separate loadable segments after adjacent ones are merged. Plain firmware needs one.
- File offsets and sizes must fit the build's address width: 64 KB on small parts, 16 MB on large ones.
- Flash segments must arrive in ascending address order, because the page cache only moves forward. avr-gcc output does. A read-modify-write option would lift this. PROPOSED
- The workspace of the load thread is 377 bytes on parts with 128-byte pages and 525 on parts with 256-byte pages: an XMODEM block, one flash page, the stream and 26 bytes of bookkeeping.
- A module is linked for one slot, and modules, their process types and their strings must lie in the first 64 KB of flash: `process_new()` reads a process type through a near pointer.

6 Technical design

6.1 Where the pieces live

The loader is part of the library. It follows the conventions of the files around it: a `// file:` line on top, a block comment that states what the unit is independent of, settings in a `*.conf.h` with `#ifndef` defaults, `CC_*` macros, a selector header in `sys/` for platform code in `cpu/avr/`, `ptstate_t` results and `ERR_*` codes.

```
src/sys/elf.h  elf.c  elf.conf.h          the stream; also builds on the host                257 lines
src/sys/elf/load.h  load.c               the load thread: XMODEM-CRC, stream, page cache     276
src/sys/bootrom.h  src/cpu/avr/bootrom.S what a running system can ask of the bootloader     138 + 41
src/sys/flash.h   src/cpu/avr/flash.*    program a page: SPM, bootloader table, do_spm       259
src/sys/module.h  module.conf.h          module descriptor, MODULE(), slots, module table    309
src/mod/loader.h                         the loader process                                 296
src/mod/shell.h                          a command line that starts, connects and loads     568
boot/boot.c  boot.conf.h  softuart.h     the bootloader, and build.sh that builds it        457
src/sys/lc-counter.h                     continuation points numbered from 1 (6.10)         147
ld/module.ld                             linker script for modules
tools/module.mjs  tools/talk.mjs         Node 22: build a module for a kernel; talk and send 180 + 230
src/mod/port.h  tools/port.mjs           the small parts: a port on the part, the shell on the PC
```

Two kinds of code, because an Arduino library is compiled apart from the sketch that uses it. The stream, the load thread and the flash driver are library sources: they are compiled with the defaults of their `*.conf.h`, and a sketch that does not use them pays nothing, because the linker leaves them out. Module slots and the loader process depend on settings only the sketch knows (how many slots, where, which load thread), so they are headers that the sketch includes after its settings, like the process types of `src/mod/`. A sketch in the Arduino IDE can therefore load modules without a change to the library.

The Arduino build compiles every source file under `src/`. The bootloader has its own entry code and no C runtime, so it stays outside that tree. It is one translation unit: `boot.c` includes `sys/elf.c`, `sys/elf/load.c`, `cpu/avr/flash.c` and the library's own `cpu/avr/uart0.c`, so that the compiler sees the stream, its sink and the drivers together.

6.2 The stream

```
/* Start a stream. Flash bytes outside [lo, hi) are refused before anything reaches the sink. */
void      elf_stream_init(elf_stream_t *s, elf_addr_t lo, elf_addr_t hi);

/* Push the next len bytes of the file. Returns PT_WAITING to be fed more, PT_ENDED when every
   loadable byte has been delivered, or an error code. */
ptstate_t elf_stream_write(elf_stream_t *s, const uint8_t *src, uint8_t len);

/* The sink: store one byte. Returns ERR_SUCCESS to continue, or the error the stream ends with. */
uint8_t   elf_stream_sink(elf_stream_t *s, uint8_t space, elf_addr_t addr, uint8_t byte);
```

The stream answers in the vocabulary of the framework: `PT_WAITING` while it wants more, `PT_ENDED` when it is done, and above that an error from `errors.h`. A protothread can pass its result straight on.

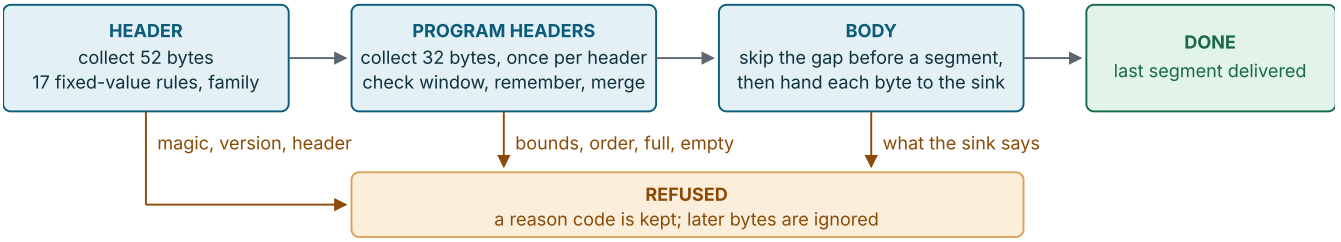


Figure 5. The states of the stream. Nothing reaches a sink before BODY, which is why a refusal in the first two states changes nothing.

Header rules. Seventeen bytes of the ELF header must each have one exact value: the magic, the class, the byte order, the file type, the machine, the position and size of the program headers. They are checked by a loop over a 34-byte table of offset and value pairs, which is smaller than seventeen comparisons and easy to extend.

What is remembered. The data of a segment arrives after all the headers, so each loadable segment is remembered as three numbers: the count of file bytes to skip before it, the address of its next byte, and the bytes still to deliver. Storing the gap and not the absolute offset removes the need to count file position at all. A segment that continues the previous one, both in the file and in memory, can be merged into it

(`ELF_CONF_COALESCE`); `.text` and `.data` always do, so plain firmware is then one remembered segment. Merging does nothing else: it saves an entry of six bytes, nine on parts above 64 KB, and costs 62 bytes of code. A kernel, where RAM is short, merges. The bootloader does the opposite: it leaves merging out and keeps four entries, which hold code, initial data and two further blocks, the same files as three entries with merging. A file with four flash segments loaded; one with five was refused at its fifth header (T28, T29). MEASURED

Layout. The scratch area for header bytes is the first member of `elf_stream_t`, so that header bytes and the hot control fields are all within the 63-byte reach of one pointer. On AVR that matters more than it looks: see 6.10.

What building it taught

- **A refused file must not cost the installed image.** The first version erased the record when the first block arrived. A wrong file then left a working board without a valid image. Invalidation now happens with the first page write, after the headers are accepted. Test T7 guards it.
- **Above 64 KB, tables in flash need the far read.** In the ATmega2560's boot section the rule table sits beyond the reach of `LPM`. The loader refused every file until the table was read through `ELPM`.
- **Code in the boot section owns no RAM once the system runs.** The thread and the page service work on the caller's workspace and buffer alone.
- **A table of relative jumps does not span an 8 KB boot section.** The service table holds addresses. Without an explicit keep, the linker's garbage collection also dropped the table and its services without a word.
- **The event queue belongs to everybody.** The first loader process posted itself an event on every turn. Under load the queue of the scheduler filled, and the event that should have told the kernel “the module runs” was dropped without a trace. The loader now keeps at most one event of its own in the queue.
- **`PT_RAISE()` takes effect on the next schedule.** A thread that is only scheduled when there is news for it would send its refusal a second late, and a process that raises waits for an event that may never come. The load thread raises and runs its handler within one call; the loader process asks for a turn before it raises (7, finding 21).
- **Byte counts move in both directions.** A check that looked cheaper where a page is committed came out 16 bytes larger than the same check per byte. Every change was measured, and three were taken back.

6.3 Flash driver

The sink for flash is a cache of one page. A byte for a different page first commits the cached one. The cache starts as all `0xFF`, so a partly covered page is padded. Committing a page is one call, and who carries it out is a setting:

```
bool flash_is_writable(void);                                    /* is the selected driver present? */
void flash_page_write(flash_addr_t base, const uint8_t *data);   /* erase one page and program it */
```

`FLASH_CONF_DRIVER`	Who executes SPM	Used by	Status
`FLASH_DRIVER_SPM`	This code: erase, fill the page buffer, write, re-enable the read-while-write section, inside `CC_ATOMIC_RESTORE()`. Written in assembly by default (`FLASH_CONF_SPM_ASM`), 76 bytes; the C version, 130 bytes, stays as the reference and passes the same tests.	the bootloader	MEASURED
`FLASH_DRIVER_BOOTROM`	The protoduino bootloader, through its service table (6.7), also where the table lies above 128 KB	a kernel	MEASURED

`FLASH_CONF_DRIVER`	Who executes SPM	Used by	Status
`FLASH_DRIVER_OPTIBOOT`	Optiboot 8 or later, through `do_spm`: erase, fill word by word, write. Parts up to 128 KB; above, the build stops with `#error` until the call has been made and tested there.	a kernel	MEASURED
`FLASH_DRIVER_AUTO`	Whichever of the two is installed, found out at run time: the service table first, then Optiboot. The default, so that a sketch needs no setting. 272 bytes against 60 and 162 for the two it combines. Above 128 KB it is the service table alone.	a kernel	MEASURED
`FLASH_DRIVER_NONE`	Nobody: the build cannot change its flash	–	–
NVMCTRL	tinyAVR 0/1/2, AVR Dx: another mechanism, not designed here	–	–

The lock bits must leave SPM writes to the application section enabled, or no bootloader of any kind can work.

6.4 The load thread

The thread speaks XMODEM with 16-bit CRC. It is decades old and exactly right for this job.

Property	Why it matters here
Built into terminal programs on every platform	The unmodified `.elf` can be sent with no tool of ours installed.
The receiver sets the pace: one block, one acknowledgement	The thread programs pages while it handles a block and acknowledges afterwards. No flow-control lines are needed, and the sender is silent while a page is programmed with interrupts off.
Blocks are numbered	A block repeated after a lost acknowledgement is recognised and not applied twice.
CRC-16 per block	The same CRC routine also protects the image record and the image, so there is one checksum routine in the loader.

A block is `SOH`, the block number, its complement, 128 data bytes and two CRC bytes. The thread holds a block in its workspace until the CRC has passed, and only then pushes it into the stream, so damaged bytes never reach flash. One extension is used: after cancelling with two `CAN` bytes the thread sends the reason code. Terminal programs ignore it; the `pd` tool prints it.

The mailbox. The thread touches no hardware. Whoever schedules it puts each received byte into a one-byte mailbox, tells it when the line has been silent for about a second, and takes up to three reply bytes out. The four calls are named after the serial layer they stand in for:

```
elf_load_init(&load, lo, hi, ELF_LOAD_MODULE, tries);   /* window, kind of file, probes to send */

if (elf_load_write_available(&load))
  elf_load_write8(&load, byte);                         /* a byte from the sender               */
elf_load_timeout(&load);                                /* or: the line is silent               */

state = elf_load_thread(&load);

while (elf_load_read_available(&load))
  send(elf_load_read8(&load));                          /* its answer, if it has one            */
```

The thread, as it stands in `src/sys/elf/load.c`. Judging a frame is a plain function, `elf_load_frame()`, which answers like the stream does: `PT_WAITING` to go on, `PT_ENDED` for a complete file, `PT_EXITED` when nobody started to send, or an error.

```
PT_THREAD(elf_load_thread(struct elf_load *self))
{
  struct pt *pt = &self->pt;

  PT_BEGIN(pt);

  self->seq = 1;
  self->end = 0;
  elf_stream_init(&self->stream, self->lo, self->hi);

  do
  {
    // Collect one frame: a block that starts with SOH, or the single byte
    // that is not the start of one, or nothing when the line stays silent.
    for (self->fill = 0; self->fill != ELF_LOAD_BLOCK_SIZE; )
    {
      uint8_t event;

      ELF_LOAD_WAIT_RX(pt, self);     // PT_WAIT_UNTIL(pt, flags & (ELF_LOAD_RX | ELF_LOAD_SILENT))
      // take what the mailbox holds: a byte, or the news that there is none
      event = self->flags & (ELF_LOAD_RX | ELF_LOAD_SILENT);
      self->flags ^= event;
      if (!(event & ELF_LOAD_RX))
        break;
      self->block[self->fill++] = self->rx;
      if (self->block[0] != ELF_LOAD_SOH)
        break;
    }

    // Judge it. The answer for the sender is in the mailbox afterwards.
    self->result = elf_load_frame(self);
    if (self->result >= PT_ERROR)
      ELF_LOAD_RAISE(pt, self, self->result);
    if (self->result == PT_EXITED)
      PT_EXIT(pt); // nobody is sending
  }
  while (self->result != PT_ENDED);

  PT_CATCHANY(pt)
  {
    // Refused: cancel the transfer, tell the sender why, and end with the
    // error that was raised.
    self->tx[2] = ELF_LOAD_CAN;
    self->tx[1] = ELF_LOAD_CAN;
    self->tx[0] = self->result;
    self->txn = 3;
    PT_THROW(pt, self->result);
  }

  PT_END(pt);
}
```

The thread returns	Meaning	Protothreads v2 construct
`PT_WAITING`	Between two bytes: schedule it again when the mailbox has news, or simply on every pass	`PT_WAIT_UNTIL`, one wait point in the whole thread
`PT_ENDED`	The file is in flash; `self->entry` is its entry point	flow reaches `PT_CATCHANY`
`PT_EXITED`	Nobody started to send within `tries` probes	`PT_EXIT`
an error	The file was refused; the sender was told why	raise, `PT_CATCHANY`, `PT_THROW`

Two details. All state is in `struct elf_load`, whose first member is `struct pt`, as the layout rule of `process.h` asks; the only local variable, `event`, does not live across the wait. And `ELF_LOAD_RAISE()` is `PT_RAISE()` with one difference: it sets the error with `LC_RAISE()` and then calls the thread again at once, so the handler runs within the same call. `PT_RAISE()` itself returns `PT_WAITING` and lets the handler run on the next schedule, which for this thread would be the next byte or a second of silence.

The scheduler of a system with one thread is the loop in `boot/boot.c`. It uses the polled calls of the library's own `cpu/avr/uart.h`:

```
static CC_NO_INLINE ptstate_t boot_schedule(struct elf_load *self)
{
  uint16_t idle = 0;
  ptstate_t state = PT_WAITING;

  do
  {
    if (uart0_rx_is_ready())
    {
      elf_load_write8(self, uart0_rx_read8());
      idle = 0;
    }
    else
    {
      __asm__ __volatile__ ("wdr");   // for a part whose fuses keep the watchdog running
      _delay_loop_1(BOOT_CONF_IDLE_DELAY);
      // 65,536 idle passes in a row: nobody is sending
      if (++idle != 0)
        continue;
      elf_load_timeout(self);
    }

    state = elf_load_thread(self);

    while (elf_load_read_available(self))
      boot_tx_write8(elf_load_read8(self));
  }
  while (PT_ISRUNNING(state));

  return state;
}
```

The listing leaves out the `?` option. One thing in the bootloader does not go through the library: opening the port. `uart0_open()` works out its divisor at run time with 32-bit divisions, which cost 292 bytes, and enables the receive interrupt; `uart0_close()` ends with `sei()`. A bootloader knows its baud rate when it is built, polls, and must not enable interrupts, so it sets the four registers itself and clears `UCSR0B` before it starts the image. Both are in section 7 (findings 2, 7 and 20).

Limits. XMODEM cannot tell the sender to stop early. Sending a stripped file removes the wasted time; a sender that stops after the last loadable byte would do the same for any file. In simulation at 115200 baud a block took 13.7 ms: 0.23 s for example 12 stripped, 3.18 s with its debug information. MEASURED

6.5 Image record and memory map



ATmega328P with the 2 KB boot section, not to scale. Window 30,592 bytes; bootloader 1,930 to 2,020 bytes used; service table in the last twelve bytes. The tests put two 2 KB module slots at 0x6000 and the module table at 0x7000.

Figure 6. Flash layout. The record page sits directly below the loader and outside the window.

Bytes	Field	Content
0 – 1	Magic	`'p' 'D'`
2 – 3	Entry	Word address at which the image is started
4 – 6	Image end	End of the written image, rounded up to a page
7 – 8	Image CRC	CRC-16 (XMODEM polynomial) of flash from 0 to the image end
9 – 10	Record CRC	Stored so that the CRC over all eleven bytes is zero

An erased page fails both the magic and the CRC, so “no record” needs no flag. The record after test T1 read `70 44 00 00 80 01 00 13 80 c1 aa`: entry 0, image end `0x000180`, image CRC `0x8013`. The record belongs to the bootloader. A kernel's identity for modules is computed by the kernel itself (6.8), so that it also exists under Optiboot, which keeps no record.

The watchdog and the reset cause

```
BOOT_RESET_CAUSE = MCUSR;           /* GPIOR0: where the image reads why the part was reset */
MCUSR = 0;                          /* WDRF has to go before the watchdog can be changed    */
BOOT_WDT = _BV(WDCE) | _BV(WDE);    /* the timed sequence ...                               */
BOOT_WDT = _BV(WDP3) | _BV(WDP0);   /* ... off; where a fuse keeps it on, 8 s               */
```

These statements run as soon as the stack is set, and the scheduling loop executes `WDR` on every idle pass for a part whose fuses keep the watchdog running: 22 bytes together. The image starts with the watchdog off, `MCUSR` at zero and the cause in `GPIOR0`; in the tests it read `01` after power-on and `08` after a watchdog reset. MEASURED

Optiboot uses the watchdog for four things. READ

1. **As its only timer.** Its receive routine is a loop that waits for a byte and has no timeout. The watchdog, armed at one second and reset with every good byte, is the timeout, and it costs one instruction.
2. **As its way out.** It does not jump to the application. It lets the watchdog reset the part, sees the cause on the way back in, and starts the application before it has touched a register. The application finds the part as a reset leaves it.
3. **To decide whether to run at all.** Only a reset through the pin makes it wait. After power-on, a brown-out or a watchdog reset the application starts at once.
4. **Because it has to.** The cause must be cleared and the watchdog switched off, or the part is reset every 16 ms.

This bootloader takes the last two. The fourth is not a choice: it is the 22 bytes above, on in every build. The third is `BOOT_CONF_WINDOW_AFTER`, 20 bytes, with Optiboot's rule as the default: wait only after a reset through the pin. The first does not fit the protocol. XMODEM is driven by the receiver: after a silence the receiver has to send its probe or its negative acknowledgement again, so it needs a clock that ticks without resetting the part, and a reset in the middle of a transfer would throw away a session that a negative acknowledgement recovers today. That clock is the idle count of the scheduling loop. The second is reached another way. What a reset would restore and the bootloader has changed is four UART registers, so it writes their reset values back before it jumps, 12 bytes (`BOOT_CONF_UART_RESET`). The image then finds the part as a reset leaves it, with three stated differences: the watchdog is off, `MCUSR` is zero, and `GPIOR0` holds the cause.

6.6 Parts without a boot section PROPOSED

On a classic ATtiny, reset always goes to address 0, and nothing protects the loader's pages. The loader sits at the top of flash and makes four provisions.

1. **The reset vector is redirected.** When page 0 of an image passes through the page cache, the loader keeps the image's own start address for the record and replaces the first instruction with a jump to itself. The other vectors stay the image's; the loader uses no interrupts.

2. **Erase runs from the top down, page 0 last.** Before the first page is written, all application pages are erased starting just below the loader. While page 0 still holds the old redirected vector, a reset reaches the loader through it. Once page 0 is erased, everything below the loader is erased, and erased flash executes as a harmless skip instruction until the program counter runs into the loader. This closes the window urboot's documentation warns about.
3. **A landing pad.** Because that skip instruction may skip one word, the loader starts with a one-word no-op.
4. **Its own serial port.** The ATtiny85 has no USART. `boot/softuart.h` provides the polled part of `uart0` on two pins, under the same names, so the bootloader source is the same on every part.

The loader's own pages remain unprotected against a runaway application. On these parts the design is an updater, not a guaranteed recovery path, and an ISP programmer remains the last resort.

The ATtiny85 build (1,958 bytes, 313 bytes of RAM, with `BOOT_CONF_VIRTUAL_RESET` and `BOOT_CONF_SOFT_UART`) compiles and has not executed. Points 2 and 3 were checked on the ATmega328P in simavr, which executes an erased word as the same skip instruction a real AVR does. Started in erased flash below the loader, at either word alignment and with the skip taken or not, execution arrived in the loader all four times. In a control build without the landing pad, one of the four cases skipped the loader's first instruction and ran its rule table as code. MEASURED

6.7 The service table

The last twelve bytes of flash describe what the boot section offers to the running system. `src/sys/bootrom.h` is the whole contract.

Offset	Bytes	Content
+0	4	Address of `elf_load_thread()`; 0 when the bootloader was built without `BOOT_CONF_LOAD_SERVICE`
+4	4	Address of `flash_page_write()`
+8	1	Version of the table: 2
+9	1	`sizeof(struct elf_load)` as the bootloader was built, in units of 4 bytes: 95 on the ATmega328P, which is 380 bytes
+10	2	`'p' 'D'`

```
bool      bootrom_is_present(void);       /* a protoduino bootloader with a table?        */
uint8_t   bootrom_version(void);
bool      bootrom_has_load_thread(void);  /* was it built with the thread?                */
uint16_t  bootrom_load_size(void);        /* bytes of workspace its thread works on       */
ptstate_t bootrom_load_thread(struct elf_load *self);            /* one turn      */
void      bootrom_page_write(bootrom_addr_t base, const uint8_t *data);

/* in the loader process: is there a thread to schedule? */
return bootrom_is_present()
    && bootrom_version() == BOOTROM_VERSION
    && bootrom_has_load_thread()
    && bootrom_load_size() <= sizeof(loader_workspace);
```

What is shared. The system and the bootloader are built separately, so they agree on the head of `struct elf_load` only: the protothread state, the mailbox, the window, the result, the entry point and the key of 6.8. Those fields have the same layout in every build for one part. Everything behind them is private to whoever runs the thread, and the table says how much room that takes. The system brings a workspace at least that large; the loader process adds a margin of 16 bytes to its own `sizeof` for a bootloader that was built with more options.

Rules. Code reached through the table uses the caller's memory and stack only. `flash_page_write()` switches interrupts off while it works, because the caller's interrupt handlers live in the section being programmed, and restores the caller's interrupt state on return. The thread itself runs with the caller's interrupts enabled and calls nothing outside the boot section.

Why addresses and not jumps. The first version of the table held relative jumps. In an 8 KB boot section a relative jump at the end of flash cannot reach a function near the start of the section, and the link failed; and link-time relaxation is free to shorten a long jump and shift whatever follows it. Plain addresses have neither problem.

Above 128 KB. The two services are called through the header and never fetched as pointers, because a pointer cannot hold their address everywhere. On an ATmega2560 a function pointer has 16 bits and reaches the first 128 KB, and the 4 KB boot section lies at `0x3F000`. The first version called through a truncated pointer; on the simulated part the call ended up in the wrong place and the page stayed unwritten. **MEASURED** The usual answer is to set `EIND` around an indirect call. That is wrong here: the load thread runs with interrupts enabled, and protoduino's UART handlers call their callbacks through pointers, so a handler that fires while `EIND` is set would jump into the wrong half of flash. The header therefore does it without `EIND`:

```
; ptstate_t bootrom_far_load_thread(struct elf_load *self, uint32_t word_address)
;   self is in r24-r25 where the thread expects it; the address arrives in r20-r23
push r20          ; the address goes on the stack as a return address ...
push r21
push r22
ret               ; ... and RET takes all three bytes of it
```

The service then returns straight to whoever called the routine. The address is the last argument, so the arguments of the service are already in the registers it expects them in, and the routine for `flash_page_write()` is the same four instructions with other registers. Both are written at file level and only declared to the compiler: as a C function without a body the first attempt was taken for a function that never returns, and the code after the call was dropped. On parts up to 128 KB the header makes a plain indirect call. The bootloader itself contains one indirect jump, the `IJMP` into the image, which clears the upper address bits by definition.

Cost and evidence. On the ATmega328P the page service adds 12 bytes to the bootloader and the thread another 76: the table, the lower bound of the window and the check of the key. In the previous draft the blocking session cost 396 bytes to export, because it had to exist twice. An application installed through the bootloader programmed a page through the table and read it back, on the ATmega328P (T10) and on the ATmega2560 for a page below and one above 64 KB (T20, T21). A kernel had modules loaded through both entries on the ATmega328P (M1, M3, M9) and on the ATmega2560 (M10, M11). **MEASURED** One limit applies: simavr does not enforce the rule that SPM must run from the boot section (S2 in 8.1), so these runs show the calling convention and not the hardware permission.

6.8 Modules

A module is an executable linked into a flash slot. Its first bytes are a descriptor, and the ELF entry point is the descriptor's address.

```
typedef struct module {
  uint16_t              magic;      /* MODULE_MAGIC, 'p' 'M'                              */
  uint8_t               abi;        /* MODULE_ABI                                         */
  uint8_t               flags;      /* reserved, 0                                        */
  uint16_t              kernel_id;  /* kernel build the module is linked for: the key     */
  const process_type_t *type;       /* the process type the module adds                   */
} module_t;
```

Anatomy of a module

This is the module of the tests, complete (`prototype/kernel/count_module.c`). It keeps a counter in its process struct, writes to the serial device process of the kernel through a pipe, and takes turns through the scheduler of the kernel. Nothing in it says that it is loaded later, except the last line.

```
#include <protoduino.h>
#include <sys/process.h>
#include <sys/process/serial.h>
#include <sys/module.h>

PROCESS_DEFINE(count, "count", 3,
  uint8_t n;                      /* all state lives in the process struct */
  char    text[4];
);

PROCESS_THREAD(count, ev, data)
{
  process_count_t *self = PROCESS_SELF(count);

  PROCESS_BEGIN();

  PROCESS_SET_PIPEOUT(&serial_process0_proc.tx_pipe);

  for (self->n = 0; self->n < 3; self->n++) {
    self->text[0] = ' ';
    self->text[1] = 'm';
    self->text[2] = (char)('0' + self->n);
    PROCESS_PIPE_WRITE(self->text, 3);
    process_poll(&serial_process0_proc.base);

    /* take the next turn after everybody else */
    process_post(pt_process, EVENT_INTERVAL, NULL);
    PROCESS_WAIT_EVENT();
  }

  PROCESS_END();
}

MODULE(count);                    /* descriptor: magic, ABI, kernel identity, &process_type_count */
```

```
# a module for one kernel build and one slot: tools/module.mjs does these steps
node tools/module.mjs kernel.elf count_module.c 0

# 1. what the kernel offers: every function and variable with a name of its own, as addresses
PROVIDE(process_post = 0x12f4);  PROVIDE(ipc_pipe_write = 0x1722);  ...     > kernel.ld

# 2. the identity of the build (CRC-16 of its image) and the slot address, read from kernel.elf
avr-gcc -Os -mmcu=atmega328p -nostartfiles -nostdlib \
        -Wl,-T,ld/module.ld -Wl,-T,kernel.ld -Wl,-e,module_descriptor \
        -Wl,--defsym=__module_base=0x6000 -Wl,--defsym=__module_kernel_id=0xa981 \
        -o count-slot0.elf count_module.c -lgcc
```

The result is a file with one flash segment of 237 bytes at the slot address, its entry point on the descriptor, and calls into the kernel as plain absolute `call` instructions. **MEASURED** `ld/module.ld` turns static variables into a link error: *module has static variables: keep state in the process struct*. The first module build hit exactly that: `process.h` declared `process_current` with `CC_EXTERN`, which is empty in C, so every C file that included it defined the variable (7, finding 10, fixed for this variable). The symbols are given as `PROVIDE()`, so a name the module defines itself is its own. The tool takes the name of a header of `src/mod/` as well and writes the three lines around it: the modules of the examples are loadable as they are (`upper` 303 bytes, `vcc` 645, `ticker` 480, `blink` 211 on the ATmega328P). **MEASURED**

Three levels of module support

The sketch that is the kernel chooses one of three levels with `MODULE_CONF_SUPPORT`, before it includes `sys/module.h` or `mod/loader.h`. The figures are the flash the test kernel grows by from one level to the next, on an ATmega328P; they include the few lines with which that kernel reports what it did.

Level	The kernel can	It brings	Flash
0. None	Nothing with modules	–	0 B
1. Modules in flash	Start the modules its module table lists, programmed with the firmware or loaded earlier. It cannot load.	`sys/module.h`: identity, table, check, start, and the functions it keeps for modules	1,236 B
2. Run time, thread of the bootloader	Also load while running, on a protoduino bootloader that offers its thread	`mod/loader.h`, and `flash.c` of the library	+3,298 B
2. Run time, own thread	Also load while running, on any bootloader that can program a page for it	The same, with `elf.c` and `load.c` of the library	+1,440 B

Of level 1, about 400 bytes are functions the test kernel would not otherwise contain: a kernel keeps the functions a module is likely to call (6.8, binding). The functions of the two headers themselves take 1,296 bytes in the kernel of M1, the loader thread 940 of them. At level 2 the kernel also holds 389 bytes of RAM for the workspace of the thread, 48 per slot for module state, and the loader process. MEASURED

Where it makes no sense. The level defaults to none, and should stay there on parts with less than 16 KB of flash or 1 KB of RAM, which covers the ATtiny25, 45 and 85; asking for run-time loading where the workspace cannot fit should be a build error. PROPOSED The levels themselves were all run (M1 to M4, M8). MEASURED

The loader process

`src/mod/loader.h` is a process type in a header, like the modules beside it. `loader_start(slot, line)` starts it and connects it to a device in both directions with `process_connect()`; when it ends it gives the input of that device back to whoever read it before, so a shell can start it on its own line. Its thread, shortened by the comments:

```
PROCESS_BEGIN();

if (!loader_has_thread() || !flash_is_writable())  /* the bootloader's, or the kernel's own copy */
  LOADER_RAISE(ERR_INIT_NOSUP);
if (self->slot >= MODULE_CONF_SLOTS || module_is_installed(self->slot))
  LOADER_RAISE(ERR_STATE_LOCKED);

if (!pt_process->has_pipein || !pt_process->has_pipeout)    /* started without a line */
  LOADER_RAISE(ERR_PIPE_BROKEN);

elf_load_init(load,
              MODULE_SLOT_ADDR(self->slot),
              MODULE_SLOT_ADDR(self->slot) + MODULE_CONF_SLOT_SIZE,
              ELF_LOAD_MODULE,
              LOADER_CONF_PATIENCE);
load->key = module_kernel_id();
timer_set(&self->silence, clock_from_seconds(1));

do {
  if (!elf_load_write_available(load)) {
  } else if (ipc_pipe_available(PROCESS_PIPEIN()) > 0) {
    ipc_pipe_read(PROCESS_PIPEIN(), &self->byte, 1);           /* the next byte from the sender */
    elf_load_write8(load, self->byte);
    timer_restart(&self->silence);
  } else if (timer_expired(&self->silence)) {
    elf_load_timeout(load);                                    /* or the news that there is none */
    timer_restart(&self->silence);
  }
```

```
    self->state = (uint8_t)loader_thread(load);

    while (elf_load_read_available(load) > 0) {                 /* its replies go out */
      self->byte = elf_load_read8(load);
      PROCESS_PIPE_WRITE(&self->byte, 1);
    }

    if (PT_ISRUNNING(self->state) && ipc_pipe_available(PROCESS_PIPEIN()) == 0) {
      if (!self->ticking)                                       /* everybody else runs */
        self->ticking = process_post(pt_process, EVENT_INTERVAL, (process_data_t)pt_process);
      PROCESS_WAIT_EVENT();
    }
  } while (PT_ISRUNNING(self->state));
```

When the thread has stopped, the loader looks at how it ended, turns the file in the slot into a running process, and cleans up after itself:

```
  if (self->state == PT_EXITED)
    LOADER_RAISE(ERR_CONN_TIMEOUT);
  if (self->state != PT_ENDED)
    LOADER_RAISE(self->state);

  self->state = module_check(self->slot);         /* the file is in the slot: is it a module for us? */
  if (self->state != ERR_SUCCESS)
    LOADER_RAISE(self->state);

  module_table_write(self->slot, true, loader_workspace.bytes);
  if (self->notify != NULL)
    process_post(self->notify, EVENT_MODULE_INIT, module_start(self->slot));
  else
    module_start(self->slot);
  process_post(pt_process, PROCESS_EVENT_EXIT, NULL);   /* brings us back for PT_FINALLY */

  PT_CATCHANY(&pt_process->pt)
  {
    self->state = (uint8_t)PT_ERROR_STATE;          /* the scheduler passes it to the error logger */
    process_post(pt_process, PROCESS_EVENT_EXIT, NULL);
    PT_THROW(&pt_process->pt, self->state);
  }

  PT_FINALLY(&pt_process->pt)
  {
    if (self->line != NULL && self->previous != NULL)   /* give the line back */
      process_connect(self->line, self->previous);
  }

  PROCESS_END();
```

The loader is an ordinary citizen. It reads `PROCESS_PIPEIN()`, writes with `PROCESS_PIPE_WRITE()`, keeps time with `sys/timer.h`, reports success with `EVENT_MODULE_INIT` and failure by ending with an error, which `process.c` hands to the error logger like the error of any process. Three lines in it work around the scheduler as it is today: the event it posts to itself before it ends, the single `ticking` event, and the poll inside `LOADER_RAISE()` (7, findings 14, 15 and 21).

The `ticking` event carries the process as its data. The first version posted it without, and worked until the kernel got a tick (`sys/tick.h`), which posts the same event number to every process: the loader took each tick for its own event and posted another, the queue filled within eight ticks, and from then on nobody could post, the loader's own report included. A process that posts an event to itself has to be able to tell it from everybody else's. **MEASURED**

Binding to the kernel

Scheme	How a module finds kernel functions	Cost on the device	Trade-off
Exact built	Linked against the addresses of one kernel build. The descriptor carries that build's identity, and the load thread refuses a module that names another.	None. No symbol table, no names, no table of entries.	Modules must be re-linked after any kernel change. A stale module is refused, not a crash.
Table option	The kernel exposes an append-only table of jumps at a fixed address; modules link against the table slots.	Four bytes per exported function, and one extra jump per call.	Modules survive kernel updates while the table only grows.

What a kernel offers. A module can only call what the kernel contains, and a build contains what somebody in it refers to. With link-time optimisation, which the Arduino IDE uses, that is stricter still: a function the kernel calls in one way only may survive in a form shaped for that call. `sys/module.h` therefore keeps a table that refers to the functions a module is likely to need (events, polls, `process_new()`, `process_connect()`, the pipe functions, `micros()`), and a sketch adds its own with `MODULE_CONF_API`. A second record holds where the slots are, so that the host tool reads slot addresses, part and clock from the kernel file and asks for nothing. A module that calls something the kernel does not offer fails at link time with the name of the function. MEASURED

The identity is a CRC-16 that `module_init()` computes over the kernel's own flash image; the host computes the same value from the kernel's ELF file and passes it to the module's link. It does not come from the bootloader's image record, because Optiboot keeps none.

Checked before the first write. The kernel hands its identity to the load thread as `self->key`. In `ELF_LOAD_MODULE` mode the sink compares the two bytes that arrive for offset 4 of the window, where a descriptor carries `kernel_id`, and refuses the file with `ERR_INIT_DEPENDENCY`. Those bytes are in the first page, which is still in the cache at that moment, so a module for another kernel build changes nothing (M5: 0 bytes programmed). The previous draft could only refuse such a module after it had overwritten the slot. MEASURED

Slots, the module table and the life cycle

- **Slots.** `MODULE_CONF_SLOTS` ranges of `MODULE_CONF_SLOT_SIZE` bytes from `MODULE_CONF_BASE`, above the kernel image and below the image record. The tests use two slots of 2 KB at `0x6000`. A module is linked for one slot; with 4.8, any.
- **The module table.** One page behind the last slot: a magic and, per slot, the identity of the kernel its module belongs to. It is written after the module has been loaded and checked, so a load that was cut short leaves nothing that could be started.
- **Load.** The thread programs the module's pages into the slot. The loader process then calls `module_check()`, `module_table_write()` and `module_start()`, which is `process_new()` with the type from the descriptor and RAM from the slot's block.
- **Start-up.** `module_autostart()` starts every slot whose table entry names this kernel. No loader is involved, which is all that level 1 needs.
- **Remove.** `module_table_write(slot, false, ...)` and `process_destroy()`: the shell's `del`. The flash of the slot is left as it is; a new module is accepted once the process of the old one has ended. MEASURED

Who programs the pages

Bootloader	How the kernel gets a page programmed	Status
protoduino bootloader	Its thread does, or its page service does for the kernel's thread	MEASURED
Optiboot 8 or later	`do_spm`: erase, fill word by word, write	MEASURED
None; part without a boot section	The kernel executes SPM itself: `FLASH_DRIVER_SPM`	PROPOSED
Any other on a megaAVR	Not possible. `loader_is_available()` says so; level 1 still works.	MEASURED M7

On Optiboot

A developer who prefers Optiboot keeps it. The firmware is uploaded the usual way, and the protoduino kernel loads modules itself, with `LOADER_CONF_THREAD` set to `LOADER_THREAD_OWN` in the sketch. The flash driver finds Optiboot by itself. What Optiboot offers was read from its source (version 8.3): the second word of the bootloader is a jump to `do_spm(address, command, data)`, which performs one SPM step and re-enables the read-while-write section after an erase or a write; a build without the feature has a `ret` there instead. `flash_is_writable()` recognises a usable Optiboot by a major version from 8 up to 127 in the last byte of flash, which erased flash is not, and a jump in that second word. It takes Optiboot's start as 512 bytes below the end of flash, as Optiboot's own helper header does. Run against the prebuilt 8.3 image from the Optiboot repository, a module was loaded cooperatively, started, entered into the table and started again after a reset (M2). MEASURED

What the tests are and are not

The kernel in M1 to M12 is `prototype/kernel/kernel.c`: a sketch with `setup()` and `loop()` on `process.c`, `ipc.c`, the serial device process, the serial layer and the UART driver of the library, built without the Arduino core. A stand-in of 40 lines supplies `main()`, timer 0 and `micros()`. Its kernel process doubles as the error logger of the scheduler, which is how it learns that a load failed.

The kernel in U3 and U4 (9.2) is a sketch of the examples, `42-shell-loader`: a shell, two modules and two slots. Built with the Arduino builder against the Arduino core, and with avr-gcc alone, it was uploaded through the bootloader inside protosim and took modules over its command line. Still open:

- Hardware.
- The ATmega1284P kernel, which was not built.
- A second serial device as the line of the loader. The loader no longer names one, but only `serial_process0` was used.

Rules and hazards

- No static variables. No interrupt handlers: the vector table belongs to the kernel.
- The slots and the module table lie in the first 64 KB of flash, also on a larger part: the scheduler reads a process type and its name with a 16-bit address and calls the thread through a 16-bit pointer. A configuration that puts them higher is a build error.
- A process struct is at most 255 bytes, because `process_type_t.size` is one byte, and at most `MODULE_CONF_RAM_SIZE`.
- A callback a module registers with the kernel, such as a UART receive handler, points into the module. It must be removed in the module's `PT_FINALLY` block. The kernel cannot check this.
- On a chip, programming a page stops everything for 7.4 to 9.0 ms with interrupts off: the datasheet gives 3.7 to 4.5 ms each for the erase and the write. One timer 0 overflow per millisecond is lost in that time, so `micros()` falls behind by that much per page.
- While a block is on the wire, events wait behind the polls of the serial device (7, finding 14).
- The reserved byte of the descriptor is where a signature over the module would be announced. It is reserved, not designed.

6.9 Compile-time configuration

Configuration follows the pattern protoduino already uses in its `*.conf.h` files: every setting has a default and can be overridden. Three rules apply. Derive from the part wherever possible: address width from `FLASHEND`, expected core family from `__AVR_ARCH__`, page size from `SPM_PAGESIZE`. An option that is off costs nothing. And fail the build, not the device: a missing setting is an `#error`, and a bootloader that outgrows its boot section fails at link time.

Setting	Default	Effect	Flash, 328P
`ELF_CONF_ADDR_BITS`	from the part	16 or 24 bits for offsets and addresses	see 6.10

Setting	Default	Effect	Flash, 328P
`ELF_CONF_MAX_SEGMENTS`	3; bootloader 4	Separate segments remembered; 6 or 9 bytes of RAM each	–
`ELF_CONF_ARCH_CHECK`	on	Refuse files for another core family	12 B
`ELF_CONF_COALESCE`	on; bootloader off	Merge adjacent segments into one entry (6.2)	+62 B
`BOOT_CONF_IMAGE_CRC`	on	Store the image CRC in the record	14 B
`BOOT_CONF_WATCHDOG`	on	Clear the reset cause, switch the watchdog off, hand the cause to the image in `BOOT_RESET_CAUSE` (`GPIOR0`)	22 B
`BOOT_CONF_WINDOW_AFTER`	2: the reset pin only	Reset causes after which the bootloader waits for a sender: 7 also power-on and brown-out, 15 every reset (5.3)	20 B
`BOOT_CONF_UART_RESET`	on	Put the UART registers back to their reset values before the image starts	12 B
`BOOT_CONF_PAGE_SERVICE`	off	Offer `flash_page_write()` to the running system	+12 B
`BOOT_CONF_LOAD_SERVICE`	off	Offer the load thread too; implies the page service, `ELF_CONF_WINDOW_LO` and `ELF_LOAD_CONF_KEY`	+90 B
`ELF_CONF_SIGNATURE`	off	Compare the device signature	+50 B
`ELF_CONF_EEPROM`	off	Write the EEPROM image	+68 B
`BOOT_CONF_INFO`	off	Answer `?` with a line that names the bootloader	+94 B
`BOOT_CONF_VERIFY`	off	Re-check the image CRC at every start	+48 B
`BOOT_CONF_VIRTUAL_RESET`, `BOOT_CONF_SOFT_UART`	off	Part without a boot section (6.6); serial on two pins	tiny only
`BOOT_CONF_SECTION`	2 KB; 4 KB above 64 KB of flash	Bytes of boot section the fuses reserve; `BOOT_CONF_START` follows from it	–
`BOOT_CONF_START`, `BOOT_CONF_WINDOW`, `BOOT_CONF_BAUD`	section, 3, 115200	Where the bootloader is linked; probes before the image starts; line speed	–
`LC_INCLUDE`	`lc-counter.h` in the bootloader	Continuation points numbered from 1 and not by source line (6.10)	−14 B
`FLASH_CONF_DRIVER`	bootloader table	Who programs a page: 6.3	60 to 160 B
`FLASH_CONF_SPM_ASM`	on	SPM driver: program a page in assembly; off selects the C reference	−54 B
`MODULE_CONF_SUPPORT`	0	Kernel: 0 none, 1 modules in flash, 2 run-time loading	see 6.8
`MODULE_CONF_SLOTS`, `_SLOT_SIZE`, `_BASE`, `_RAM_SIZE`	2, 2048, 0x6000, 48	Kernel: where modules go and how much RAM each gets	–
`LOADER_CONF_THREAD`	bootloader's	Kernel: `LOADER_THREAD_ROM` or `LOADER_THREAD_OWN`	see 6.8
`LOADER_CONF_PATIENCE`	30	Kernel: seconds to wait for a sender	–

Each figure is the difference from the default bootloader of 1,930 bytes when that one setting is flipped.

MEASURED

Filling the boot section

The fuses reserve a boot section of one of four sizes, and whatever is not used is wasted. A build step therefore starts from the smallest bootloader and adds options in order of priority for as long as the result still fits the chosen block. The order is itself a setting. The default puts the checks that prevent damage first and the services for the kernel next: family check, page service, load thread, image CRC, wait after the reset pin only, UART reset, segment merging, signature, EEPROM, description line, CRC check at start.

Part	Block	Used	Spare	What the block then holds
ATmega328P	0.5, 1 KB	–	–	Nothing: the smallest bootloader is 1,872 bytes
	2 KB	2,020 B	28 B	Family check, page service, **load thread**, image CRC, wait after the reset pin only, UART reset. The standard section for this part.
	4 KB	2,334 B	1,762 B	All eleven options
ATmega1284P	1, 2 KB	–	–	Nothing: the smallest bootloader is 2,278 bytes
	4, 8 KB	2,950 B	1,146 B, 5,242 B	All eleven options. 4 KB is the standard section for this part.
ATmega2560	1, 2 KB	–	–	Nothing: the smallest bootloader is 2,278 bytes
	4, 8 KB	2,950 B	1,146 B, 5,242 B	All eleven options. 4 KB is the standard section for this part.

These are the results of the step as built (`fit.sh`). MEASURED Three things follow. On the ATmega328P the 2 KB block holds the default bootloader and lends its load thread to the kernel, with 28 bytes to spare. What it leaves out is segment merging, which the fourth table entry makes up for (6.2). On the parts above 64 KB the bootloader does not fit the 2 KB block, so their standard section is 4 KB: that is the default of `BOOT_CONF_SECTION` there and the block `fit.sh` fills when it is given none, and it holds every option. The decision costs 2 KB of a 128 or 256 KB part and buys the device-signature check, the EEPROM image, the description line and the CRC check at every start, all of which were run in that configuration (T15 to T19, T25, M10). And the space that is left, 1,146 bytes, is where the next features belong: a verify mode, signed images, and the fix-up stream of 4.8. On parts without a boot section the bootloader simply ends on a page boundary, so there is no block to fill.

6.10 Measured budgets

Build	Flash	RAM
ATmega328P, smallest	1,872 B	377 B
ATmega328P, default	1,930 B	377 B
ATmega328P, with the load thread offered	2,020 B	379 B
ATmega328P, everything on	2,334 B	383 B
ATmega1284P, default	2,338 B	525 B
ATmega2560, smallest	2,278 B	525 B
ATmega2560, default	2,338 B	525 B
ATmega2560, everything on	2,950 B	532 B
ATtiny85, serial on two pins BUILT	1,958 B	313 B

Where the 1,930 bytes go	Flash
Load thread: frames, XMODEM, replies	438 B
Stream: delivery loop and flash sink	358 B
Bootloader: record, start decision, image start	364 B
Stream: program headers	180 B
Page cache and page programming	160 B
Stream: header check, rule table, init	198 B
Scheduling loop and `uart0` calls	146 B
CRC-16; entry code with the watchdog	88 B

RAM. The bootloader's RAM is the workspace of the thread, and nothing else: on the ATmega328P the XMODEM block (133), the page buffer (128), the stream with its four segment entries (90) and 26 bytes of mailbox, window and bookkeeping. It is not cleared at reset; every field is set before it is read, so the bootloader has no `.bss` and no code to clear one.

What the protothread costs. The blocking bootloader of the previous draft was 1,640 bytes; the protothread one was 2,014, and after the assembly and the numbered continuation points described below it was 1,938. Since then it gained 56 bytes that neither draft had, for the watchdog, the start decision by reset cause and the UART reset (6.5), and lost the 62 of segment merging (6.2): 1,930. The 374 bytes are the price of a thread whose state lives in memory and not in registers: every wait stores a continuation and returns, every resume goes through a `switch`, and every variable is a load and a store. Offering the load to the kernel costs 2,020 bytes against 2,036, and the new one also checks the kernel identity and takes care of the watchdog: the thread is compiled once, where the blocking session had to exist twice. The comparison at the level that matters:

ATmega328P, 2 KB boot section	Previous draft, blocking	This draft, protothread
Bootloader that also loads modules for the kernel	2,036 B with merging; 12 B spare	2,020 B with watchdog care and start by reset cause, four entries for merging; 28 B spare
The system during a load	stands still for the whole transfer	runs: 15,113 heartbeat turns in 350 ms
A module for another kernel build	overwrites the slot, then is refused	refused before the first page
Kernel code for run-time loading	a call, in a stand-in kernel	1,296 B in the sketch and 272 B of flash driver, on the real scheduler

What would shrink it. Nothing has been tried from this list, in order of expected yield:

1. A hand-written per-byte path for the stream and the frame check. It would live beside the C version, which the host build needs: a last resort.
2. Replace the 128-byte block with a smaller frame, which also suits the small ATtinys.
3. Build with a newer avr-gcc. The repository's `platformio.ini` already points at one; only 7.3.0 was available here.

What already mattered, in the order it was found. The first version of the protothread bootloader was 2,520 bytes. One wait point in the thread with a plain function that judges a frame, a mailbox the thread never has to wait on, and setting the UART registers directly (286 of the bytes) brought it to 2,094. `-fno-move-loop-invariants -fno-ivopts -fno-tree-scev-cprop` saved 26. A workspace that needs no clearing, and with it no `.bss` code, saved 48. The assembly described next saved 62, and numbered continuation points 14. `-mstrict-X` keeps the workspace pointer out of the X register. MEASURED

Numbered continuation points. `lc-switch.h` identifies a wait point by the source line it stands on, so the number is stored and compared as 16 bits as soon as the line is above 255. That made the size of the thread depend on where it stands in its file: six comment lines added above it moved one wait point past line 255, and the bootloader grew by two bytes. `src/sys/lc-counter.h` is the same header with the points numbered 1, 2, 3 from the compiler's `__COUNTER__`; the bootloader selects it through `LC_INCLUDE`, which `lc.h` already provides for. It saves 14 bytes on the ATmega328P and 18 on the ATmega2560, and comments are free again. MEASURED

What Optiboot does in assembly. Optiboot 8.3 is one C file of 1,689 lines, and its ATmega328 image is 474 bytes. It is C almost throughout: ten inline-assembly statements and a header that re-issues avr-libc's SPM sequences with `OUT`. Its size comes less from assembly than from what it leaves out: no file format, no checksum, no record of what is installed; the host decides and the device stores pages. It uses assembly where the compiler cannot say a thing at all or says it badly. READ

What was taken from it. Erasing and programming a page was 130 bytes from the compiler; with `OUT` to the control register and one routine that issues every SPM and waits, it is 76, and those 54 bytes put segment merging back into the 2 KB section of the ATmega328P. `IJMP` into the image, in place of an indirect call followed by a loop that is never reached, saved 8. The assembly is confined to `src/cpu/avr/flash.c` and one line of `boot/boot.c`: the stream stays in C because it also builds on the host, the load thread because it is the part that should read like protoduino. Optiboot's `-fno-split-wide-types` changed nothing. MEASURED

Time. Line time dominates: in simulation a block took 13.7 ms at 115200 baud on the ATmega328P, 15.1 ms on the ATmega2560 and 14.0 ms through the loader process, about 9 KB per second. A chip adds the time to program each page (S5 in 8.1).

7 Fixes for current protoduino caveats

Findings 1 to 9 came out of reviewing the UART and serial layers as a base for the loader. Findings 10 to 21 came out of building the loader process and a module on the scheduler, the pipes and the serial device process; finding 22 was reported by the review of the Protosim MCP design and reproduced here. Each was checked by running protoduino's real sources in simulation, except where marked as read. Three have been applied since, when the loader moved into the library: 4, 17, and 10 for the one variable that stood in the way. Their rows say so. The others stand. The ones a user is most likely to meet are 14 and 15 in the scheduler, 11 to 13 in the serial path, and 10 with a newer compiler.

#	Where	What happens	Evidence
1	`src/cpu/avr/uart_private_impl.h:69`	`uartN_baudrate()` reports twice the real rate whenever the double-speed bit is off. The line rate itself is correct.	**MEASURED** request 57600: registers give 58,823, function returns 117,647. Request 300: real 300, reported 600.
2	`uart_private_impl.h:81–87`	`uartN_close()` ends with an unconditional `sei()`. Called with interrupts disabled, it switches them on under the caller.	**MEASURED** interrupt flag 0 before the call, 1 after.
3	`src/sys/serial/serial_private_impl.h:293`	`serialN_flush()` tests the interrupt flag the wrong way round compared with its own documentation. With interrupts off it enables the transmit interrupt, which cannot fire, and sends nothing.	**MEASURED** ten bytes queued with interrupts off: after 60,000 calls nine are still unsent. With interrupts on all ten go out.
4	`docs/loader.md`	Describes a loader that cannot work on AVR (module code in RAM, `-fpic`) and an API that is not in `src/`. The README links to it as experimental. **Fixed:** the file is now the manual for modules, the shell and the loader as built.	**READ** avr-gcc prints "-fpic is not supported". No loader source exists in the tree.
5	`uart_private_impl.h:105–109`	The frame-error branch in `uartN_rx_error()` can never run: the flag is masked out three lines earlier. Skipping frame errors is a documented choice; the dead branch is the only defect.	**READ**
6	UART header documentation	Nothing says that the polled receive calls and the receive interrupt must not be used together. With both active the stream splits between them.	**MEASURED** input `hello`: the callback received `hlo`, the poller `el`.
7	`uart_private_impl.h`	The interrupt handlers and callback pointers are always compiled in. A bootloader that wants the same register mapping without vectors cannot have it.	**READ** The polled calls themselves work with interrupts off **MEASURED**: input `hello` echoed in full.
8	`platformio.ini`	`lib_deps` is an absolute path on one Windows machine, so the project builds nowhere else as checked in.	**READ**
9	`uart_private_impl.h:44, :162`	A baud rate of 0 divides by zero in `openex()`. A `#pragma message` prints on every build of every UART instance.	**READ**

#	Where	What happens	Evidence
10	`src/cc.h`, `CC_EXTERN`; `sys/process.h`	`CC_EXTERN` is `extern "C"` in C++ and nothing in C. `CC_EXTERN struct process *process_current;` and `PROCESS_EXTERN()` therefore define the variable in every C file that includes the header. avr-gcc 7 merges the copies silently; from GCC 10 on, the link fails. A module got its own `process_current` this way, in RAM it is not allowed to have. It also turned out that no file defined the variable at all: the copies were its only definition. **Fixed for this variable:** declared `extern`, defined in `process.c`. `CC_EXTERN` itself and `PROCESS_EXTERN()` are as they were: making the macro `extern` in C stops the build of `sys/serial.h`, which combines it with another storage class.	**MEASURED** `nm` shows a common symbol; with `-fno-common`: “multiple definition of `process_current'”
11	`src/sys/serial.h:6–7`, `serial.conf.h`	`serial.h` includes `protoduino-config.h` before `cc.h`, so `serial.conf.h` chooses the buffer sizes before `RAMEND` is defined. Undefined, it counts as 0, and every part gets the 16-byte buffers meant for parts with less than 1 KB of RAM.	**MEASURED** ATmega328P: both rings 16 bytes; 64 when `avr/io.h` is included first
12	`src/sys/process/serial_private.h`, service loop	On a poll the serial device moves as many bytes from `tx_pipe` as the UART ring has room for, then waits for the next event. Nothing polls it for the rest, so they stay in the pipe until somebody happens to write or receive again.	**MEASURED** 80 bytes written at once: 15 left the UART
13	`src/sys/process/serial.h`, usage text	The documented way to choose a baud rate, setting `baud` and then calling `process_new()`, has no effect: `process_new()` clears the struct. The device opens at the default 9600.	**MEASURED** 115200 requested: `UBRR0L` = 207, which is 9600
14	`src/sys/process.c`, `process_run()`	`if (do_poll()) return; do_event();` dispatches an event only when no poll was pending. A process that polls itself on every turn stops all events for good. And because the serial device polls on every received byte, events also stop while bytes arrive back to back.	**MEASURED** a self-polling process ran; a second process never got its INIT event. During a transfer a heartbeat waited 15.0 ms, the length of a block; 2.0 ms with the change below.
15	`process.c`, `call_process()`	When a process ends, the scheduler arms `PT_FINAL` and waits for the next invocation to run the `PT_FINALLY` block and unlink it. Nothing causes that invocation: `process_poll()` ignores a process in that state. The process stays linked, its cleanup never runs and its pool slot is never free, unless an event happens to be addressed to it.	**MEASURED** a process that ended: after 2.5 s its cleanup had not run and its state was still EXITING
16	`process.c`, `process_start()`	The return value of the `process_post()` that delivers the INIT event is ignored. With the default queue of three usable entries, a fourth process started in a row is linked into the list and never runs.	**READ**

#	Where	What happens	Evidence
17	`src/sys/clock.h`	Accepts only the ATmega328, 168 and 88 families and stops with `#error` on every other part, although `cpu/avr/clock.h` has nothing in it that depends on the part. Timers, and everything that uses them, do not build for the ATmega2560 that example 22 targets. **Fixed:** the header accepts every AVR.	MEASURED "clock: Unsupported Platform!" for `-mmcu=atmega2560`
18	`src/sys/pt/pipe.h:11–17`	`PT_STDIN()` and `PT_STDOUT()` yield the pipe struct, not its address, so `PT_PIPE_AVAILABLE()`, `PT_PIPE_READ()` and the others do not compile with the `struct ipc_pt` they are written for. The `_EX` forms, which processes use, are fine.	MEASURED "incompatible type for argument 1 of 'ipc_pipe_available'"
19	`serial_private.h`, `_on_rx`	The receive callback of the serial process returns nothing, but is installed where the serial layer expects a function that returns whether it has consumed the byte. The layer acts on whatever is left in the result register: sometimes it stores the byte in its own ring as well, which nobody reads, and counts overflows there.	MEASURED compiler: "incompatible pointer type"; effect READ
20	`uart_private_impl.h`, `openex()`	The divisor and the reported baud rate are computed at run time with 32-bit divisions, also when the rate is a constant. A program whose only UART call is `uart0_open(115200)` pays 292 bytes for it.	MEASURED `uart0_openex` 224 B, `__udivmodsi4` 68 B
21	`sys/pt.h` `PT_RAISE()` with `sys/process.h`	`PT_RAISE()` returns `PT_WAITING` and lets the handler run the next time the thread is scheduled. A process is only scheduled when an event or a poll arrives, so a process that raises may never reach its `PT_CATCHANY` block, and its error is never reported. There is no `PROCESS_RAISE()`.	MEASURED the first loader process: a refused module produced no report until its raise was preceded by a poll
22	`process.c`, `do_poll()` and broadcast	The sweep advances through `pp->next` after it has dispatched to a process. If that process finished on this dispatch it has been unlinked, its `next` is cleared, and the sweep ends there. Processes behind it keep their pending poll, and nothing requests another sweep. With modules that can end and be removed, this decides whether the others still run.	MEASURED two processes polled together, the first ended on its poll: the second was never polled and stayed pending

Proposed changes

```
/* 1. uart_private_impl.h, openex(): remember which divisor is in force */
  uint8_t  samples = 8;                                   /* double speed: clock / 8 */
  uint16_t baud_setting = (F_CPU / 4 / baud - 1) / 2;
  __UCSRA__ = 1 << __U2X__;
  if (((F_CPU == 16000000UL) && (baud == 57600)) || (baud_setting > 4095)) {
    baud_setting = (F_CPU / 8 / baud - 1) / 2;
    __UCSRA__ = 0;
    samples = 16;                                         /* normal speed: clock / 16 */
  }
  ...
  CC_TMPL_VAR(baudrate) = F_CPU / ((uint32_t)samples * (baud_setting + 1));

/* 2. uart_private_impl.h, close(): leave the caller's interrupt state alone */
void CC_TMPL_FN(close)(void)
{
  CC_ATOMIC_RESTORE()
  {
    CC_TMPL_VAR(baudrate) = 0;
    __UCSRB__ &= ~(_BV(__RXEN__) | _BV(__TXEN__) | _BV(__RXCIE__) | _BV(__UDRIE__));
  }
}

/* 3. serial_private_impl.h, flush(): poll the port when interrupts are OFF */
  if (bit_is_clear(SREG, SREG_I))
  {
    if (!CC_TMPL2_FN(tx_is_ready)())
      return cnt;
    CC_TMPL2_FN(tx_write8)(ringb8_get(&VAR_TX));
    return cnt - 1;
  }
  CC_TMPL2_FN(tx_enable_int)();
  return cnt;
```

```
/* 10. cc.h: a declaration must be a declaration in C as well */
#ifdef __cplusplus
#define CC_EXTERN extern "C"
#else
#define CC_EXTERN extern
#endif
/*     and process.c then needs the one definition:  struct process *process_current; */

/* 11. sys/serial.h: the configuration needs the device header first */
#include <cc.h>                     /* brings in avr/io.h: RAMEND, RAMSTART */
#include <protoduino-config.h>

/* 12. sys/process/serial_private.h: keep going while tx_pipe holds bytes */
      CC_TMPL2_FN(flush)();
      if (ipc_pipe_available(&self->tx_pipe) > 0)
        process_poll(pt_process);

/* 14. sys/process.c: polls first, then one event in the same call */
void process_run(void)
{
  do_poll();
  do_event();
}

/* 15. sys/process.c, call_process(): a process that ended gets the turn its cleanup needs */
    PT_FINAL(&p->pt);
    process_post(p, PROCESS_EVENT_EXIT, NULL);

/* 22. sys/process.c, do_poll(): read the successor before the dispatch can unlink */
  for (struct process *pp = process_list, *next; pp != NULL; pp = next) {
    next = pp->next;
    ...

/* 19. sys/process/serial_private.h: say that the byte has been taken */
static bool CC_CONCAT2(CC_TMPL_PREFIX, _on_rx)(uint_fast8_t ch)
{
  uint8_t b = (uint8_t)ch;
  ipc_pipe_write(&CC_CONCAT2(CC_TMPL_PREFIX, _proc).rx_pipe, &b, 1);
  process_poll(&CC_CONCAT2(CC_TMPL_PREFIX, _proc).base);
  return true;
}

/* 21. sys/process.h: raise in a process, and get the turn in which the handler runs */
#define PROCESS_RAISE(err)                \
  do {                                    \
    process_poll(pt_process);             \
    PT_RAISE(&pt_process->pt, (err));  \
  } while (0)
```

- **4.** Replace `docs/loader.md` with a short page that points here, and mark the README entry as a design, not a feature.
- **5.** Delete the dead branch. If frame errors are wanted later, report them from the receive interrupt, where the flags are read before the data register and are valid; put it behind `UART_CONF_REPORT_FRAME_ERRORS`, default off, so behaviour stays as it is.
- **6.** One paragraph in `uart_private_header.h`: use the callbacks with interrupts on, or the polled calls with interrupts off, never both.
- **7.** Add `UART_CONF_ISR`, default 1. With 0, the handlers, the callback setters and the receive-interrupt enable in `openex()` are left out, and the bootloader can use the same template.
- **8.** A relative library reference in `platformio.ini`.
- **9.** Return `ERR_SERIAL_BAUD` for a zero rate; put the message behind a verbose-build switch.

- **13.** Correct the usage text: set `baud` after `process_new()`, before the scheduler runs; or let `process_new()` leave the bytes behind `struct process` alone when the type asks for it.
- **14, with 12.** With polls no longer ahead of events, a device may poll itself until its work is done without stopping anybody. The measurement of 4.6 is this change and nothing else: `prototype/kernel/build_kernel.sh` with `FAIR=1` builds against a copy of `process.c` in which only `process_run()` differs.
- **16.** Let `process_start()` return whether the INIT event was queued, and `process_new()` return NULL when it was not.
- **17.** Select `cpu/avr/clock.h` on `__AVR__`, as `sys/uart.h` does for its driver.
- **18.** `#define PT_STDIN(pt) (&(pt)->pipein)` and the same for `PT_STDOUT`.
- **20, with 7.** A compile-time form: `#define UART_UBRR(baud) ((F_CPU / 4 / (baud) - 1) / 2)` and `uartN_open_ubrr(ubrr, flags)` that only writes registers. With the polled calls as `static inline` in the header, a bootloader could then open, poll and close the port through the library and pay nothing for it. The bootloader of this document would lose its one direct register access.

The loader works on the library as it is. It carries a workaround for 14, 15 and 21, marked in `src/mod/loader.h`, and the test sketch carries one for 12 and 13. The tick (`sys/tick.h`, `tick_run()` in `loop()`) is the general form of the workaround for 12, 15 and for timers: one event for every process each 16 ms, so that whoever waits for a timer, for room in a pipe or for its own end gets a turn.

Findings from my first review that testing withdrew

They are listed so the record is straight. The polled receive calls do work after `open()` when interrupts are off. `rx_clear_errors()` does clear an overrun under simavr (S7 in 8.1); on a real chip those flags are read-only and clear when the data register is read, so the call is harmless there. The transmit path through the data-register-empty interrupt is sound: 18 bytes went out in 19 handler calls. The 57600 special case is the deliberate Arduino-compatible behaviour. The read-modify-write on the control register can at worst cause one extra interrupt that switches itself off.

8 Simulator findings: simavr and protosim

Running the prototype and the probes turned up behaviour in both simulators that matters to anyone testing firmware with them. This section lists all of it: where simavr differs from a real chip, defects in protosim, and what protosim would need to test this loader without a separate harness. The versions are protosim `6fadc6f` and simavr `a6fc4cc`, the commit protosim pins. The tables describe those versions. Since then protosim `f4d382e` records reset causes and fixes B2, B6, B9 and B13, and `21fd8b2` keeps the last flash byte that simavr's loader drops (S1): without it a kernel inside protosim does not find the service table, whose last letter is that byte. `8097fee` and `c336482` give it the outside world of a small part (a serial line on two pins, levels and waves into pins, voltages, a log of output pins) and correct S15. simavr is unchanged. Recorded probe output is in `prototype/results-simulators.txt`.

8.1 simavr: where the simulation differs from the hardware

#	Where	What simavr does	A real AVR	Evidence
S1	`sim_elf.c:103, :255`	The firmware loader is given the last valid flash address as its limit, so a chunk that ends on the last byte of flash is loaded one byte short, with only a log line. This is an off-by-one. The harness here puts the byte back, so the tests run the real layout; protosim does the same since `21fd8b2`.	The chunk fits.	**MEASURED** "Loading 4 bytes of Flash data at 7ffc would overflow buffer (32767 bytes)"
S2	`avr_flash.c`, SPM handling	SPM is carried out wherever the program counter is.	On parts with a boot section, SPM outside it does nothing.	**MEASURED** a program at address 0 erased and wrote a page
S3	`avr_flash.c`	A page write replaces the page with the buffer contents.	Programming only clears bits; without an erase the result is old AND new.	**MEASURED** `F0` then `3C` with no erase read back `3C`, not `30`
S4	`avr_flash.c`	There is no read-while-write state. The application section reads back correctly straight after a page write, and the busy bit is never set.	The section cannot be read until it is re-enabled.	**MEASURED** data read back at once; `SPMCSR = 04`
S5	`avr_flash.c`	Erase and write finish within the instruction. The page-write bit stays set in `SPMCSR` afterwards.	Each takes milliseconds; the control bits clear on completion.	**MEASURED** zero busy polls
S6	`sim_core.c:777`	When the program counter reaches the end of flash, the CPU is marked as crashed. Erased words themselves are executed correctly, as the skip instruction.	The program counter wraps to address 0.	**MEASURED** "CPU CRASHED" after running through erased flash with nothing above it
S7	`avr_uart.c:334–336`	Any write to the UART status register clears the overrun flag; the line that would preserve it is commented out.	The flag is read-only and clears when the data register is read.	**MEASURED** status `AA` before `rx_clear_errors()`, `A2` after
S8	`avr_uart.c`, status read	The frame-error flag is recomputed only while a received byte is waiting, and writes cannot clear it, so it stays set after the last byte has been read.	The flag belongs to the byte that is next to be read.	**READ** It matches the note in protoduino's driver that the flag cannot be cleared.
S9	`avr_uart.c`, input queue	The receiver queues 63 bytes before it reports an overrun.	Two bytes and the shift register.	**MEASURED** 63 bytes drained after 100 were sent with nobody reading
S10	`avr_uart.c`, receive timer	With more than one byte queued, the receive flag stays set while the interrupt is raised again only one byte time later. Code that polls with interrupts on sees the flag and can take the byte ahead of the handler.	The interrupt is taken as soon as the flag is set.	**MEASURED** 568 consecutive polls saw the flag at 9600 baud; none at 19200

#	Where	What simavr does	A real AVR	Evidence
S11	`sim_avr.c`, `avr_reset()`	A reset discards every pending cycle timer and restarts the cycle counter at zero. An external reset is not recorded in `MCUSR`, and the register is cleared at every reset, where a chip keeps the flags firmware has not cleared. protosim `f4d382e` records the cause itself and keeps the flags, also across a watchdog reset.	Reset causes are all recorded.	**MEASURED** `MCUSR` = `01` at power-on and `08` after a watchdog reset, both correct. Timers and external reset **READ**; effects in B6 and B7.
S12	`cores/sim_mega2560.h`	The ATmega2560 core has no signature row. The header reads `avr_flash.h` before the device header, so the bit that selects the row is unknown when the core is declared. Reading the signature from software returns flash contents.	Returns `1E 98 01`.	**MEASURED** the read returned `0C 72 0C`, bytes 0, 2 and 4 of the program. The ATmega328P and ATmega1284P cores return their signatures.
S13	`avr_flash.c:77`	The signature and fuse rows are selected by Z with `RAMPZ` added. With `RAMPZ` not zero the read falls through to flash contents.	The rows are read with LPM, which takes its address from Z alone.	**MEASURED** ATmega1284P: `1E 97 05` with `RAMPZ` = 0, `0C 46 0C` with `RAMPZ` = 1
S14	`avr_watchdog.c`, reset	After a watchdog reset the watchdog is enabled again with the prescaler at zero, but it does not fire after 16 ms: in a probe the next reset came after 0.28 s and later each time, and under the bootloader after about 2 s.	The next reset comes 16 ms later unless software clears the cause and switches the watchdog off.	**MEASURED** 28, 29, 31, 32, 34, 36 ticks of 10 ms between resets
S15	`avr_extint.c`, `avr_ioport.c:28`	The flag of an external interrupt cannot be cleared by writing a one to it: on the megaAVR parts the write sets it, on an ATtiny, where it shares `GIFR` with the pin change flag, the write is dropped. Firmware that polls `INTF0` sees an edge on every look. protosim corrects this since `c336482`.	Writing a one clears the flag.	**MEASURED** `GIFR` read 0x40 for ever; the keeper of 9.2 never noticed that its heartbeat had stopped
S16	`sim_avr.c:404`, sleep	Sleep is one mode: every interrupt wakes the part, so a timer that power-down stops on a chip wakes it in simulation. And a sleeping part is slept for in real time. protosim has `--warp` for the second since `8097fee`, and reports the share of time a part slept.	In power-down only the watchdog, the external and the pin change interrupts wake the part.	**MEASURED** a part asleep on an 8 s watchdog was awake 4.7 s of 640 s with timer 0 running, 42 ms with it stopped

#	Where	What simavr does	A real AVR	Evidence
S17	`sim_tinyx5.h`, timer 1	Timer 1 of the ATtiny25/45/85 is defined in normal mode only. With `PWM1B` set the compare output never rises, so no PWM appears at `OC1B`.	The pin carries the duty cycle in `OCR1B`.	READ, and MEASURED: the pin stayed low
S18	`avr_flash.c`, ATtiny	Clearing the page buffer (`CTPB`) is not modelled, and the `SELFPRGEN` fuse is not looked at: self-programming always works. A page sent again after a failed one may keep words of the first.	`CTPB` clears the buffer; without `SELFPRGEN` SPM does nothing.	READ
S19	`avr_uart.c:144`, status read	Each read of the UART status register with nothing pending sleeps the host (`usleep(1)`, in practice far longer), so firmware that sends with polled writes runs about 80 times slower than real time. protosim switches it off under `--warp` since `b978d65`.	A status read costs one instruction.	MEASURED the terminal UI test took more than 10 minutes unfinished, 4.4 s with `--warp`

What this means for a bootloader tested in simulation

S2 to S5 are the ones that matter most here. A loader that forgets to erase before writing, forgets to re-enable the read-while-write section, does not wait for an operation to finish, or runs outside the boot section passes every test in simavr and fails on a chip. The prototype does all four correctly, but the simulated results in section 9 are not what shows that. They establish parsing, protocol and addressing. Flash discipline, timing and power loss need hardware. S12 and S13 touch one option: the signature check cannot run on the simulated ATmega2560 at all, and on the ATmega1284P it runs only because the bootloader clears `RAMPZ` before the read, which a chip does not need.

8.2 protosim: defects

#	Where	What happens	Evidence	Suggested fix
B1	`src/protosim.c:470`	The exit status is always 0: the last line is `return force_exit_success ? 0 : 0;`. A CPU crash before `--exit-on-uart` matches is reported as success. Only running out of `--max-steps` fails.	MEASURED "CPU CRASHED", exit status 0	Non-zero on a crash, and whenever an expected string was not seen.
B2	`src/protosim.c:304`	For a bootloader given as ELF, the entry point is read from the temporary instance's program counter, which simavr never sets when it loads a file. The loader is "started" at address 0.	MEASURED `entry=0x0000` for the ELF, `0x7800` for the same loader as HEX	Fixed in `f4d382e`. Use the firmware's base address or the ELF entry point.

#	Where	What happens	Evidence	Suggested fix
B3	`src/protosim.c:305`	The ELF path copies the temporary instance's whole flash over the simulated flash, which wipes the application loaded just before. The HEX path overlays only the bootloader. With B2 the result looks right by accident: address 0 is now erased, and erased flash runs through to the loader.	**MEASURED** only the loader's banner appears; the application's never does	Copy only the bootloader's own range.
B4	`src/protosim.c:195`	A main firmware given as Intel HEX is read with simavr's single-chunk reader. Of a file with two regions, only the first is loaded.	**MEASURED** application and loader in one file: “Loaded HEX: 180 bytes”, the loader absent	Use simavr's multi-chunk reader, or protosim's own.
B5	`src/protosim.c:84, :349`	`--uart0-in` injects one byte per millisecond, starting 1 ms after start, whatever the baud rate and whether or not the receiver is on. Bytes sent before the receiver is enabled are lost. At 9600 baud bytes arrive faster than a byte time and queue (S10).	**MEASURED** receiver enabled at 5 ms: the first 5 of 16 bytes were lost	Start when the receiver is enabled, as the TCP path does, and pace from the configured rate.
B6	`src/protosim.c:349`	Injection stops for good after any reset, because simavr's reset discards the timer that drives it (S11).	**MEASURED** after a watchdog reset none of the remaining bytes arrived	Fixed in `f4d382e`: registered again after every reset.
B7	Profiling reports	After a reset the cycle counter restarts at zero, so the report's totals no longer match the per-function sums.	**MEASURED** “hottest single edge: 200.0% of total runtime”; 99.4% without a reset	Keep a running total of its own.
B8	`src/protosim.c:384`	`--bootloader` always starts the STK500 uploader thread. No switch turns it off or selects another protocol. In headless mode, where no terminal device exists, the thread printed nothing after its start message.	**MEASURED** and **READ**	A switch; see 8.3.
B9	No assignment anywhere	simavr's `reset_pc` is never set, so a reset during a `--bootloader` run restarts at address 0 and not in the loader.	**READ** The harness set it and reset correctly.	Fixed in `f4d382e`: set for `--bootloader`, and by `--reset-vector`.
B10	`include/stk500_uploader.h:36`	The uploader's page size is fixed at 128 bytes, so the automatic upload suits ATmega328P-class parts only.	**READ**	Take it from the part.
B11	`src/ihex_loader.c:65`	The bootloader HEX reader fills gaps between regions with `00`, where erased flash is `FF`.	**READ**	Fill with `FF`.
B12	`--uart0-in`	The input is a C string. A file that contains zero bytes, such as an ELF, cannot be passed.	**READ**	See 8.3.

#	Where	What happens	Evidence	Suggested fix
B13	`src/uart_pty.c:159`	On Linux the bridge wrote to the PTY inside a trace macro that is compiled out. Nothing the firmware sent reached the PTY; input worked.	MEASURED no probe arrived in 25 s while the simulation ran at full speed	Fixed in `f4d382e`.

8.3 protosim: additions for testing this loader PROPOSED

- **Binary input.** `--uart0-in-file`, or better an XMODEM sender beside the STK500 one, with a switch for which of the two runs with `--bootloader`, and one for neither.
- **Flash comparison.** `--expect-flash <file.elf>`: at exit, compare flash with simavr's own load of that file. This is the oracle test of section 9 as a one-line check for CI, and it needs B1 fixed to be of use.
- **Reset into the loader, with a cause.** Done in `f4d382e`: `--reset-vector`, `--reset-cause`, `--reset-at <ms>[:cause]`, and on Linux and macOS `SIGUSR1` for the reset pin and `SIGUSR2` for a power cycle of a running simulation. P1 to P5 in 9.1 use it.
- **A strict flash mode.** An option that reports SPM outside the boot section, a page write with no erase before it, and reads of the application section while it is busy. That would turn S2 to S4 from blind spots into test failures. It could live in simavr or as a check in protosim.

9 Verification

9.1 End to end in simulation

The bootloader ran inside simavr. A harness sent each file with XMODEM as a terminal program would, at 115200 baud, and afterwards compared the simulated flash below the record page with what simavr's own ELF reader produces for the same file.

Test	Scenario	Outcome
T1	328P: unmodified avr-gcc ELF, 12,868 bytes, clean line	Flash identical to the reference; record written; application reached
T2	328P: block 3 damaged in transit, acknowledgement of block 5 lost	One block sent again after a negative acknowledgement, one duplicate dropped; flash identical
T3	328P: protoduino example 12, 29,624 bytes with debug information	Flash identical; 3.18 s of line time
T4	328P: the same example stripped, 2,048 bytes	Flash identical; 0.23 s
T5	328P, empty chip: an ATmega2560 file	Refused `0xD0`; flash untouched
T6	328P, empty chip: random bytes	Refused `0x51`; flash untouched
T7	328P: install, reset, then offer an ATmega2560 file	Refused `0xD0`; the installed application intact and started again 4.8 s after the reset
T8	328P, all options: file with EEPROM data and a matching signature	Flash identical; EEPROM holds `01 02 03 04 05`
T9	328P, all options: an ATmega168P file, same core family, other device	Refused `0x53` at the signature; no record written; the bootloader keeps control
T10	328P: an installed application programs a page with `flash_page_write()` through the service table	Page reads back as written; the caller's interrupt state restored
T11	2560: small image	Flash identical; record written; application reached
T12	2560: 106,830-byte image reaching far flash, 839 blocks	Flash identical; 12.7 s of line time

Test	Scenario	Outcome
T13	328P: execution started in erased flash below the bootloader, at both word alignments, with the skip taken and not taken	The bootloader was reached in all four cases. In a control build without the landing pad, one case skipped its first instruction and ran the rule table as code
T14	1284P: small image, default bootloader in the 4 KB section	Flash identical; record written; application reached
T15	1284P, every option, 4 KB section: `?` sent first; file with EEPROM data and a matching signature; then one bit of the installed image flipped and the part reset	Answered `pD2 atmega1284p`; flash identical; EEPROM written; the damaged image was **not started**, the bootloader kept probing
T16	1284P, every option: an ATmega1284 file, same core family, other device	Refused `0x53` at the signature
T17	1284P, default bootloader, which does not check at start: the same flipped bit. The control for T15	The damaged image was started after 3.8 s
T18	2560, every option, 4 KB section: `?` sent first; file with EEPROM data and no signature section (S12); then one bit flipped and a reset	Answered `pD2 atmega2560`; flash identical; EEPROM written; the damaged image was not started
T19	2560, every option: the 106 KB image; then one bit above 64 KB flipped and a reset	Flash identical; 12.8 s of line time; the damaged image was not started
T20	2560: an installed application programs a page through the service table of the 4 KB boot section, above 128 KB	Page reads back as written; interrupt state restored. With the call of the previous draft the page stayed empty
T21	2560: the same for a page above 64 KB	Page reads back as written
T22	328P: an image that arms the watchdog at 250 ms and never feeds it; bootloader built without watchdog care, as the previous draft was. The control for T23	After the first watchdog reset the image **never started again**: 15 resets in 30 s, each inside the bootloader's wait. On a chip the resets come every 16 ms (S14)
T23	328P: the same image, default bootloader	116 resets in 30 s, the image running again at once after each. It read the cause `01` at its first start and `08` after each watchdog reset
T24	328P: the same image; bootloader set to wait after every reset	The image started again after each of 8 resets, 3.8 s and three probes later every time
T25	2560, every option in the 4 KB section: the same image	115 resets in 30 s, the image running again at once after each
T26	328P: a reset with each cause and nobody sending, for the settings 2, 7 and 15 of `BOOT_CONF_WINDOW_AFTER`	The image started after 3.8 s and three probes exactly where the setting says, and at once with no probe otherwise. A jump to the bootloader was waited for under every setting
T27	328P: a running image with interrupts on, a timer interrupt and the UART at 300 baud jumps to the bootloader	The bootloader probed, took the file again and started it
T28	328P: firmware with four flash segments: code, initial data and two blocks at fixed addresses	Flash identical, up to `0x5850`
T29	328P: the same with a fifth segment	Refused `0xE0` at the fifth header; nothing written

Inside protosim

The scenarios above run in a harness on simavr. These five run the default bootloader in protosim itself, on an empty ATmega328P with its serial port on protosim's PTY. A script sends files with XMODEM as a terminal program would, and resets the part through protosim: `SIGUSR1` pulls the reset pin, `SIGUSR2` cycles the power (`run_protosim.sh`).

Test	Scenario	Outcome
P1	Empty part, power-on; a file is sent	The bootloader probed; 7 blocks acknowledged; the image started and read the cause `01`
P2	Power cycle	The image started at once, with no probe before it
P3	Reset through the pin; a second file is sent	The bootloader probed and took the file; the new image started and read the cause `02`
P4	Reset through the pin; nobody sends	Three probes, then the image again
P5	Power cycle	The new image started at once

Getting there took the reset causes in protosim and one repair: on Linux its PTY bridge did not pass on anything the firmware sent (B13).

9.2 Run-time module loading in simulation

Here the bootloader only sits in flash. The test kernel of 6.8 runs on the ATmega328P (M1 to M9) and on the ATmega2560 (M10 to M12): protoduino's scheduler with the serial device process, a kernel process, a heartbeat process and the loader process. The harness sends it a module with XMODEM while it runs. Afterwards the slots are compared with simavr's own load of the module file, the kernel image is checked for changes, and the part is reset to see what starts from flash. The kernel prints how many turns the heartbeat took during the load and the longest time it waited for one.

Test	Scenario	Outcome
M1	protoduino bootloader in the 2 KB section that offers its load thread; the loader process schedules it there	Slot identical to the reference; module started and ran as a process; entered into the module table; started from flash after a reset. 349.9 ms; heartbeat 15,113 turns, longest wait 15.2 ms
M2	Stock Optiboot 8.3 as the bootloader; the kernel links its own load thread and programs through `do_spm`	The same. 361.7 ms; 14,505 turns, 15.2 ms. The kernel finds Optiboot at run time; the same binary serves M3.
M3	protoduino bootloader with only the page service; the kernel links its own load thread	The same. 361.5 ms; 14,505 turns, 15.1 ms
M4	Kernel built for modules in flash only; module programmed together with the firmware	Module started from the module table; the kernel has no loader
M5	A module linked against another kernel build	Refused `0xA4` with **0 bytes programmed**; the kernel process got the error from the scheduler and reported it
M6	A firmware image offered to the loader process	Refused `0xA2` before any write: it lies below the slot
M7	Kernel built to load, on a bootloader that offers no service	The kernel reports that it cannot load and carries on
M8	Kernel built without module support, module present in flash	Module ignored
M9	A module runs from slot 0; a second one, linked for slot 1, is loaded beside it	Both slots identical to their files; after a reset both start and take turns
M1'	M1 against a copy of `process.c` in which `process_run()` dispatches an event after the polls	The same load: 15,691 turns, longest wait 2.0 ms
M10	2560: bootloader with every option in the 4 KB section, above 128 KB; the loader process schedules its load thread through the far call	As M1. 365.3 ms; 14,446 turns, 30.5 ms: twice the heartbeat waited out two blocks in a row
M11	2560: bootloader with only the page service; the kernel links its own load thread and has its pages programmed through the far call	As M3. 376.1 ms; 14,003 turns, 15.3 ms

Test	Scenario	Outcome
M12	2560: a module linked against another kernel build	Refused `0xA4` with 0 bytes programmed

In every load the kernel image was unchanged afterwards. The heartbeat takes fewer turns than in the previous draft because the test kernel now runs the tick and keeps the functions modules may call; the transfer times are the same. The heartbeat figures are from simavr, which programs a page in no time; on a chip each page adds 7.4 to 9.0 ms to the longest wait.

Driven from a shell

The examples of `examples/5-systems` are the same machinery in the form a user meets it: a shell (`src/mod/shell.h`) that lists processes and modules, starts, pauses, ends and connects them, and with the loader receives and removes modules. `tools/test_systems.sh` builds each system, starts it in the simulator and holds a conversation with it through `tools/talk.mjs`; every answer is checked.

Test	Scenario	Outcome
U1	`40-shell` on the ATmega328P in protosim: shell, ticker, blink, upper, vcc	Processes and modules listed; a module started, connected to the serial line and written to; a process paused, continued and ended; a connection to nobody refused. 7,430 B flash, 517 B RAM.
U2	`43-monitor`: modules chosen and connected by the sketch, the shell built to look only	States, connections and the log shown, with the warning of the process that watches the supply and the LED process it paused; every command that changes something refused. 6,990 B, 557 B.
U3	`42-shell-loader` on the ATmega328P: the bootloader with its load thread in the 2 KB section, in protosim; the sketch arrives through the bootloader, then modules through the shell	82 blocks of kernel accepted and started. `get 0`: `upper` arrives, is listed and works. `get 1`: `vcc` arrives and reports the supply. `del 1`, then a module for another kernel is refused and the log names the loader and the reason. `get` into an occupied slot refused. After a power cycle the module of slot 0 runs again. `del 0`, `get 0`: another module takes the slot. 8,738 B, 970 B.
U4	The same on the ATmega2560, the bootloader in its 4 KB section above 128 KB	The same, through the far call. 8,906 B, 1,116 B.
U5	`41-shell-tiny85` on the ATtiny85 in protosim, which bridges the serial line of two pins since `8097fee`	Processes, modules and pipes over a line made of two pins. 5,914 B, 275 B. No loading: see 6.8.

The small parts: the shell on the PC

On an ATtiny85 the shell is more than a third of the image, and the ATtiny45 and 25 have no room for it at all. The examples of `examples/6-attiny` turn the arrangement round. The part keeps a port (`src/mod/port.h`): a process of about 470 bytes that reads memory, flash and EEPROM, writes memory and EEPROM, and calls a function, each on a request of twelve bytes with a check byte. The shell runs on the PC (`tools/port.mjs`) and takes every name and address from the ELF file of the sketch: `ps` walks the process list by reading it, `stop` and `join` call the functions of the kernel. The ATmega parts keep their shell on the part.

How far the port opens is a level: 1 reads, the default; 2 also writes the settings of the sketch and runs the actions it names, which is what a finished product needs; 3 writes any memory and calls any function; 4 also programs modules into flash slots. Levels 3 and 4 open only for a minute after a reset through the reset pin, so that somebody has to be at the board, and stay open while requests keep coming. A refused request is answered with its reason.

The tick comes from the watchdog there and the processor sleeps between ticks (`sys/tick.h`, `sys/idle.h`). Modules for signals pass samples, not text, and each is a plain function on a small struct with a process around it, so that a sketch can run a stage as a process with a pipe, at about 30 bytes of RAM, or call it from one process. Settings are named bytes with built-in values that the PC can change in EEPROM (`sys/settings.h`).

Test	Scenario	Outcome
U6	`50-port` on the ATtiny85: port, ticker, blink, upper; the shell on the PC	Processes, modules and pipes listed; processes connected, paused, started, written to and ended through the port; the port itself and a name nobody has refused. 3,836 B flash, 247 B RAM, against 5,914 B with the shell on the part.
U7 U8 U9	`51-keeper` on the ATtiny85, 45 and 25: looks after another board. In protosim the board is a heartbeat on a pin that stops after 6 s and returns at 14 s, and a supply that sags to 4.0 V from 20 s to 24 s.	The reset line pulled 4 s after the last heartbeat (at 10.0 s) for 96 ms; held from 20.2 s until the supply is back at 24.1 s; pulled on request from the PC; all three counted in EEPROM; a setting changed through the port. 4,666 B and 288 B, 3,916 B and 166 B, 1,804 B and 67 B.
U10 U11 U12	`52-sentinel` : sleeps on its watchdog, records the temperature of the chip and its supply in EEPROM	Both records filled, with lowest and highest, and shown on the PC. Asleep for all but 36 to 42 ms of ten minutes at a sample every 64 s. 3,902 B and 280 B, 3,664 B and 155 B, 1,726 B and 70 B.
U16	`50-port` on the ATtiny85 after a power-on: the port at level 4, behind the guard	Reading, settings and actions work; writing memory, calling and loading are refused with the reason, until a reset through the reset pin; then open for the window, and kept open while requests keep coming.
U17	The same with a window of 3 s, and a watchdog reset after the pin reset	Closed again 3 s after the last request; the watchdog reset leaves it closed, because the port cleared the flag of the pin when it saw it.
U18	Modules into the two flash slots of the ATtiny85 through the port: the PC fills the page buffer of the chip, a few words per request, and has the page programmed; no bootloader, no RAM for a page	`upper` in 5 pages and `blink` loaded, listed, started and used; a module for a slot in use, for another slot and for another build refused; pages below the end of the sketch, of the table and beyond a page refused by the part itself; `del` frees a slot for another module; after a power cycle both slots run again. 5,151 B flash, 301 B RAM.
U13 U14 U15	`53-regulator` : `sense | smooth | curve | pwm`	1.0 V at the input gives a duty cycle of 0, 2.5 V of 60 %, 4.5 V of 100 %; a setting changed while it runs moves it to 73 %. 4,492 B and 286 B, 3,566 B and 149 B, 1,592 B and 64 B.

The three figures of U7 to U15 are for the three ways the same sketch is put together, which follow the RAM of the part: on the ATtiny85 every stage is a process and the port is there; on the ATtiny45 the stages are functions of one process, with the port; on the ATtiny25 they are functions of one process and there is no port, so its results were read from the pins and from EEPROM. MEASURED What these parts draw was not: a simulator has no current.

U3 and U4 were also run with the kernel as the Arduino builder makes it, against the Arduino core and with link-time optimisation (8,948 and 9,812 bytes): the module tool reads everything it needs from that file. Three things came out of these runs that the test kernel had not shown: the functions a module calls have to be kept on purpose (6.8), the last flash byte in protosim (S1), and the event queue that a tick and a self-posted event fill together (6.8). MEASURED

9.3 On the host

The same stream, compiled natively, was fed real files in chunks of random size and its output compared with `avr-objcopy`.

- Flash images identical for the test program, for example 12, for its stripped form, for one-byte-at-a-time feeding, and for the 106 KB image. The signature bytes extracted from the test program were those of the ATmega328P.
- Refused as designed: a file for another family (`0xD0`) and a non-ELF file (`0x51`), both within the first 127 bytes.
- The step that fills the boot section was run for every block size of three parts (6.9).

9.4 The library findings

`prototype/probes/run_probes.sh` reproduces findings 10 to 15, 17 to 20 and 22 of section 7 with small sketches on the library; its recorded output in `results-library.txt` is from the library before 10 and 17 were fixed.

9.5 Not verified

- Anything on hardware, including fuse and lock-bit settings and real programming times.
- Loss of power during an update. The transaction argument in 4.4 is by construction.
- The bootloader on an ATtiny: it has never executed. Only its run-through mechanism was checked, on the ATmega328P (T13). The shell and the appliances run on the ATtiny parts (U5 to U15); nothing is loaded there. On the ATtiny85 the port loads modules (U18); that has run in protosim only, which does not model clearing the page buffer or the `SELFPRGEN` fuse (S18).
- The ATtiny examples under an ATtiny core in the Arduino IDE: they were built with avr-gcc and a stand-in core. The current a sleeping sentinel draws. The PWM of the regulator at its pin (S17): the duty cycle given to the timer was checked.
- Flash discipline. simavr accepts a missing erase, a missing read-while-write re-enable and SPM from the application section (8.1), so the simulated results cannot show that the loader gets these right.
- The device-signature check on the ATmega2560: simavr's core for that part has no signature row (S12). The check ran on the ATmega328P and the ATmega1284P (T8, T9, T15, T16).
- The Optiboot flash driver above 128 KB of flash. The build stops with an error there.
- The watchdog on a chip: the 16 ms between resets that simavr stretches (S14), and a part whose fuses keep the watchdog on, which simavr does not model. Reset through the pin and brown-out are not recorded by simavr (S11): the harness writes the cause into `MCUSR` at each reset it makes, as protosim does.
- How long a real page write holds the system up during a load, and what that does to `micros()`.
- Module loading on the ATmega1284P.
- The host tools on a real serial port: `talk.mjs` was only used on protosim's pseudo terminal.
- Optiboot on a real board, and Optiboot versions other than the 8.3 image that was used.

9.6 Test plan from here

- **Host unit tests in CI** for the core, with the files above as fixtures, and a fuzzer on `elf_stream_write()`. The stream is small, pure C and takes untrusted input, which is the ideal fuzzing target.
- **protosim end to end** per supported part. `tools/test_systems.sh` is the start: U1 to U15 run in protosim as it is. The oracle comparison of T1 to T29 and M1 to M12 still needs the additions of 8.3 and B1 fixed.
- **Hardware**: an Uno and a Mega 2560, with an ISP programmer at hand for fuses. A relay on the supply to cut power at random points of an update, hundreds of times, checking that the board always answers afterwards.

10 Roadmap, risks and open questions

Step	Deliverable	Done when
R0	The fixes of section 7: the scheduler ones (14, 15, 16, 21) first. The clock header (17) is done.	The probes in `prototype/probes` give the corrected results, and the workarounds in `mod/loader.h` can go
R1	**Done in this draft.** The loader in the library: stream, load thread, flash driver, module support, loader process; an example sketch that loads a module. Open: host tests and fuzzing for the stream in CI.	Fixtures pass in CI on the host and for two AVR parts

Step	Deliverable	Done when
R2	The bootloader for the ATmega328P in its 2 KB section and for the ATmega2560 and ATmega1284P in their 4 KB section, with the fill step: built. protosim fixes B1 to B9 and the additions of 8.3: partly (B2, B6, B9, B13, reset causes, S1).	T1 to T29 and M1 to M12 run in protosim; first boards flashed with a terminal program
R3	Hardware: an Uno with this bootloader and with Optiboot, a Mega 2560; the power-cut rig	M1 to M3, M10, U3 and U4 pass on the boards; real page times and their effect on the scheduler are measured
R4	**Partly done.** Built: a tool that builds a module for a kernel (`tools/module.mjs`), one that talks and sends (`tools/talk.mjs`), removal and reloading of modules, shell commands for the loader. Open: PlatformIO upload through the tool, Windows, an update request from the application.	`pio run -t upload` sends the `.elf`
R5	ATtiny path on hardware; relocation by fix-up stream; signed modules; a size pass (6.10)	Each on its own evidence

Risks

- **Size.** The protothread form costs 374 bytes, of which assembly and numbered continuation points recovered 76. On the ATmega328P the 2 KB section has 28 bytes left. The parts above 64 KB use the 4 KB section, where 1,146 bytes are free.
- **The far call.** Four instructions that rely on which registers avr-gcc passes arguments in. A compiler that changes that breaks the call without a message; T20, T21, M10 and M11 would show it.
- **Page writes still stop the system.** Cooperative means between pages. Each page holds everything for 7 to 9 ms on a chip, and no AVR design can avoid that.
- **The scheduler under load.** Until finding 14 is fixed, a process that waits for an event waits out every block of a transfer.
- **ATtiny recovery.** Without hardware protection for the bootloader's pages, a faulty application can destroy it. The claim for these parts is deliberately weaker.
- **Exact binding.** Convenient while developing, awkward for distributing modules. The table scheme is the answer and costs flash.
- **Optiboot is a moving target.** The flash driver relies on where Optiboot puts `do_spm` and on its size. Both are checked at run time, but only version 8.3 was tried.
- **Old compiler.** All sizes are from avr-gcc 7.3.0. A newer one may be better or worse, and it turns finding 10 into a link error.

Open questions

Settled on 12 November 2026: the ATmega2560 and ATmega1284P take a 4 KB boot section, and the bootloader is built with every option there; no blocking build is kept for 2 KB. On the ATmega328P the 2 KB section keeps the exported load thread and gains the immediate restart after a watchdog reset; segment merging made room for it, and a fourth segment entry does its work. The bootloader waits for a sender only after a reset through the pin; protosim was given reset causes so that this can be run in simulation.

- Which small part is meant by “ATtiny8”? The answer decides whether the small-frame transport is needed early.
- Should updates be authenticated, and if so with what? A keyed hash costs several hundred bytes.
- Which binding scheme is the default for modules: exact or table?
- Should module slots be allowed above 64 KB on the large parts? Today that is a build error; lifting it needs far reads in the scheduler.
- Where should the bootloader's build live, given that it cannot be under `src/` ?

- **Unload.** Queued events hold plain process pointers, and a slot's RAM block is reused: an event still in the queue can reach the next module in that slot. The shell's `del` is built without a guard against that: a new module is only accepted once the process of the old one has ended, which leaves the window of an event posted before and delivered after. Purging the queue for a process that ends, or handles with a generation, would close it.

A ELF fields the loader reads

Where	Offset	Field	Use
ELF header	0 – 3	magic	Must be `7F 45 4C 46`
	4, 5	class, data	Must be 1 and 1: 32-bit, little-endian
	16	`e_type`	Must be 2, an executable. A relocatable `.o` is refused here.
	18	`e_machine`	Must be 83, AVR
	24	`e_entry`	Start address; for a module, the address of its descriptor
	28	`e_phoff`	Must be 52: program headers directly after the ELF header
	36	`e_flags`	Low 7 bits name the core family; compared with `__AVR_ARCH__`
	42, 44	`e_phentsize`, `e_phnum`	Must be 32; count of program headers, below 256
Program header	0	`p_type`	Only 1, a loadable segment, is considered
	4	`p_offset`	Where the segment's bytes are in the file; must not go backwards
	8, 20	`p_vaddr`, `p_memsz`	Run address and size in memory; not used
	12	`p_paddr`	Load address. Its top byte selects the memory; the rest is the address there.
	16	`p_filesz`	Bytes to deliver; a segment with none is skipped

For the ATmega328P test program avr-gcc wrote six loadable segments: code at load address 0; initial `.data` at run address `0x800100` but load address `0x136`, directly after the code; `.bss` with no file bytes; EEPROM at `0x810000`; fuses at `0x820000`; signature at `0x840000`. **MEASURED**

B Reproducing the measurements

The code is in the library (6.1), the test bench in `docs/elfloader/prototype/`. It needs `gcc-avr`, `avr-libc`, `libelf-dev`, Node.js 22 and protosim built with its simavr submodule. The Optiboot image is not included: copy `optiboot_atmega328.hex` from the Optiboot repository into `fixtures/` for M2, M4 and M8.

```
./make_fixtures.sh                      # test files, and the test kernels with their modules

# the stream on the host, against a file built with avr-gcc
gcc -std=gnu11 -O2 -DELF_CONF_EEPROM=1 -DELF_CONF_SIGNATURE=1 -DELF_CONF_MAX_SEGMENTS=6 \
    -I../../../src -o host_test host_test.c ../../../src/sys/elf.c
./host_test fixtures/app-atmega328p.elf 5 out.bin

# the harness that runs the bootloader inside simavr
S=../../../../protosim/libraries/simavr/simavr
gcc -O2 -o sim_harness sim_harness.c -I$S/sim -I$S/sim/avr \
    $S/obj-x86_64-linux-gnu/libsimavr.a -lelf -lm -lpthread

./run_tests.sh                          # T1 to T29
./run_modules.sh                        # M1 to M12
./run_protosim.sh                       # P1 to P5, in protosim itself
./fit.sh atmega2560                     # fill the standard boot section of a part
./sizes.sh                              # the tables of 6.8 to 6.10
probes/run_probes.sh                    # findings 10 to 22 of section 7
./build.sh m2560 atmega2560 default -DBOOT_CONF_LOAD_SERVICE=1
../../../tools/test_systems.sh          # U1 to U15: the systems of the examples, from the repository roo
```

Recorded outputs are in `results-sizes.txt`, `results-fit.txt`, `results-tests.txt`, `results-modules.txt`, `results-protosim.txt`, `results-library.txt` and `results-simulators.txt`, and `tools/results-systems.txt`. Findings 1 to 9 of section 7 come from the other programs in `probes/`, built against protoduino's own sources and run with `protosim --uart0-in … --uart0-out …`; the simulator findings of section 8 from `simprobes/` and `sim_harness --fallthrough`.

Sources

- Contiki, `core/loader/elfloader.c`, `symtab.c`, `cle.c`: github.com/contiki-os/contiki. Removal: github.com/contiki-ng/contiki-ng, commit 6220aea14.
- simple-avr-bootloader: github.com/m3y54m/simple-avr-bootloader
- Optiboot: github.com/Optiboot/optiboot, read at commit e363ecb: `optiboot.c` (version 8.3, `do_spm`, `pre_main`) and `examples/demo_dospm/optiboot.h`. Version 8 notes: forum.arduino.cc/t/optiboot-now-at-version-8-0/547763
- urboot, on erased flash and the first-page hazard: github.com/stefanrueger/urboot. Optiboot on ATtiny: github.com/SpenceKonde/ATTinyCore, `avr/extras/Ref_Optiboot.md`
- micronucleus and its reset-vector patch: github.com/micronucleus/micronucleus
- Zephyr LLEXT and symbol link identifiers: docs.zephyrproject.org/3.7.0/services/llext/api.html
- Microchip, ATmega48A/88A/168A/328P family, “Self-Programming the Flash” and “USART Control and Status Register 0 A”: onlinedocs.microchip.com
- avr-libc memory sections: the avr-libc user manual, “Memory Sections”
- protoduino and protosim sources at the branch named on the cover; simavr at commit a6fc4cc.